

MATRIX & GRID ALGORITHMS

A complete placement + competitive programming guide to matrix traversal, grid problems, and 2D algorithms.

1. MATRIX TRAVERSAL

1. Overview

Matrix traversal refers to systematically visiting every element of a 2D array (matrix) in a specific order. The most common traversals are **row-major** (left-to-right, top-to-bottom), **column-major** (top-to-bottom, left-to-right), and **diagonal traversal**. It is the foundation for all grid-based problems.

2. Intuition

Think of a matrix as a spreadsheet with rows and columns. A row-major traversal is how you read a book: left to right on each line, then move to the next line. Column-major is like reading a newspaper column: top to bottom, then shift to the next column.

Why it matters: Every grid algorithm (BFS, DFS, DP, etc.) builds on top of basic traversal. If you cannot traverse correctly, you cannot solve any grid problem.

Step-by-step reasoning:

1. A matrix has R rows and C columns.
2. Each cell is identified by (row, col) .
3. Traversal is controlled by nested loops.
4. The outer loop typically controls the row, the inner loop controls the column.
5. Swapping loop order changes the traversal pattern.

3. When to Use It

- Any time you need to visit all elements of a 2D array
- As a preprocessing step before applying other algorithms
- When printing, copying, or transforming a matrix
- When flattening a 2D array into 1D

Trigger phrases: "traverse the matrix", "print in order", "visit all cells", "iterate over grid"

4. When Not to Use It

- When you only need a subset of cells (use direct indexing instead)
- When the matrix is sparse (use sparse representation)
- When you need random access to specific cells only
- When you need to process in a specific non-linear order (use BFS/DFS)

5. Core Concepts

Concept	Explanation	Why It Matters
Row-major	Outer loop over rows, inner loop over columns	Default traversal; cache-friendly
Column-major	Outer loop over columns, inner loop over rows	Used when column access is prioritized
Index mapping	<code>index = row * cols + col</code> to flatten 2D → 1D	Essential for DP and memoization
Boundary checking	Verifying <code>0 <= r < R</code> and <code>0 <= c < C</code>	Prevents out-of-bounds errors
Direction arrays	<code>dr = {0,0,1,-1}, dc = {1,-1,0,0}</code>	Used for 4-directional moves

6. Step-by-Step Algorithm

Row-major traversal:

- 1. Let `R = number of rows`, `C = number of columns`.
- 2. For `i = 0` to `R-1`:
 - For `j = 0` to `C-1`:
 - Process `matrix[i][j]`

Column-major traversal:

- 1. Let `R = number of rows`, `C = number of columns`.
- 2. For `j = 0` to `C-1`:
 - For `i = 0` to `R-1`:
 - Process `matrix[i][j]`

7. Dry Run

Matrix:

```
1 2 3
4 5 6
7 8 9
```

Row-major: 1, 2, 3, 4, 5, 6, 7, 8, 9 **Column-major:** 1, 4, 7, 2, 5, 8, 3, 6, 9

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
```

```
// Row-major traversal
void rowMajor(vector<vector<int>>& mat) {
    int R = mat.size(), C = mat[0].size();
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++)
            cout << mat[i][j] << " ";
    cout << "\n";
}

// Column-major traversal
void columnMajor(vector<vector<int>>& mat) {
    int R = mat.size(), C = mat[0].size();
    for (int j = 0; j < C; j++)
        for (int i = 0; i < R; i++)
            cout << mat[i][j] << " ";
    cout << "\n";
}

// Safe traversal with boundary check
bool isValid(int r, int c, int R, int C) {
    return r >= 0 && r < R && c >= 0 && c < C;
}

// 4-directional neighbor traversal
void neighbors(vector<vector<int>>& mat, int r, int c) {
    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};
    int R = mat.size(), C = mat[0].size();
    for (int k = 0; k < 4; k++) {
        int nr = r + dr[k], nc = c + dc[k];
        if (isValid(nr, nc, R, C)) {
            cout << mat[nr][nc] << " ";
        }
    }
}

int main() {
    vector<vector<int>> mat = {{1,2,3},{4,5,6},{7,8,9}};
    rowMajor(mat);    // 1 2 3 4 5 6 7 8 9
    columnMajor(mat); // 1 4 7 2 5 8 3 6 9
    return 0;
}
```

9. Python Implementation

```
from typing import List

def row_major(mat: List[List[int]]):
    for row in mat:
        for val in row:
            print(val, end=" ")
```

```

print()

def column_major(mat: List[List[int]]):
    R, C = len(mat), len(mat[0])
    for j in range(C):
        for i in range(R):
            print(mat[i][j], end=" ")
    print()

def is_valid(r, c, R, C):
    return 0 <= r < R and 0 <= c < C

def neighbors(mat, r, c):
    dirs = [(-1,0), (1,0), (0,-1), (0,1)]
    R, C = len(mat), len(mat[0])
    for dr, dc in dirs:
        nr, nc = r + dr, c + dc
        if is_valid(nr, nc, R, C):
            print(mat[nr][nc], end=" ")
    print()

mat = [[1,2,3],[4,5,6],[7,8,9]]
row_major(mat)
column_major(mat)

```

10. Code Explanation

- **Row-major function:** Outer loop iterates rows, inner loop iterates columns. This is the natural memory layout in C++ (row-contiguous), making it cache-friendly.
- **Column-major function:** Outer loop iterates columns, inner loop iterates rows. This is cache-unfriendly for C++ but sometimes required.
- **isValid function:** Centralized boundary check prevents index errors and avoids repetitive code.
- **Direction arrays:** `dr` and `dc` encode the four cardinal directions. This pattern is used in BFS, DFS, and all grid traversal problems.

11. Complexity Analysis

Operation	Time	Space
Full traversal	$O(R \times C)$	$O(1)$
Neighbor access	$O(1)$	$O(1)$
Boundary check	$O(1)$	$O(1)$

12. Common Patterns

Pattern	Identification	Approach	Example
Flatten matrix	"Convert 2D to 1D"	<code>index = r * C + c</code>	LeetCode 54

Pattern	Identification	Approach	Example
Diagonal traversal	"Print diagonally"	Sum $r + c$ is constant on each anti-diagonal	LeetCode 498
Zigzag traversal	"Alternating direction"	Toggle direction flag per row	LeetCode 103 (binary tree)

13. Common Mistakes

- **Swapping row/column order:** Always double-check `mat[i][j]` vs `mat[j][i]`
- **Off-by-one in loops:** Use `i < R`, not `i <= R`
- **Assuming square matrix:** Check `R != C` when using column-major
- **Not checking bounds** in neighbor traversal

14. Edge Cases

- Empty matrix: `R == 0 || C == 0`
- Single row: `R == 1`
- Single column: `C == 1`
- Non-rectangular (jagged) arrays
- Very large matrix (performance matters)

15. Variations

Variation	Description	Importance
Zigzag row-major	Alternate left-to-right and right-to-left per row	Medium
Snake traversal	Same as zigzag	Medium
Spiral traversal	Outer-to-inner spiral	High (see next section)
Diagonal traversal	Top-left to bottom-right diagonals	Medium

16. Related Algorithms/Data Structures

Related	Connection
BFS/DFS on grid	Build on traversal with visited tracking
DP on grid	Traversal order determines DP correctness
Matrix multiplication	Uses specific traversal patterns
Image processing	Convolution uses traversal with windows

17. Practice Problems

Easy

- **Transpose Matrix** (LeetCode 867) — Swap rows and columns

- **Flipping an Image** (LeetCode 832) — Row reversal + inversion

Medium

- **Diagonal Traverse** (LeetCode 498) — Diagonal order traversal
- **Shift 2D Grid** (LeetCode 1260) — Grid shift with flattening

Hard

- **Range Sum Query 2D** (LeetCode 304) — Prefix sum on grid
- **Minimum Path Sum** (LeetCode 64) — DP traversal order

18. Interview Explanation

"Matrix traversal is the simplest operation on a 2D array. We use nested loops where the outer loop controls rows and the inner loop controls columns. The key insight is that the order of loops determines the traversal pattern. For any grid algorithm, I always define a helper function to check boundary conditions and use direction arrays for neighbor access. This pattern scales to BFS, DFS, and DP on grids."

19. Revision Notes

- Row-major: `for i → for j`
- Column-major: `for j → for i`
- Boundary check: `0 <= r < R && 0 <= c < C`
- Direction arrays: `dr = {-1,1,0,0}, dc = {0,0,-1,1}`
- Flatten: `idx = r * C + c`
- Complexity: $O(R \times C)$ time, $O(1)$ space

20. Final Cheat Sheet

Aspect	Details
When to use	Visit all cells, preprocessing, flattening
Main operations	Row-major, column-major, neighbor traversal
Complexity	$O(R \times C)$ time, $O(1)$ space
Key code	<code>for(i→R) for(j→C) process(mat[i][j])</code>
Edge cases	Empty, single row/col, non-square

2. SPIRAL TRAVERSAL

1. Overview

Spiral traversal visits matrix elements in a clockwise spiral pattern: top row left-to-right, right column top-to-bottom, bottom row right-to-left, left column bottom-to-top, then repeat inward. It is a classic interview problem that tests boundary handling and loop control.

2. Intuition

Imagine peeling an onion layer by layer. Each layer (or "ring") of the matrix is traversed completely before moving inward. The outer boundary is traversed first, then the next inner boundary, and so on.

Analogy: Think of a spiral staircase. You walk along the outer wall, then turn, walk along the next wall, turn again, and keep going until you reach the center.

Step-by-step reasoning:

1. Define four boundaries: `top`, `bottom`, `left`, `right`.
2. Traverse the top row from left to right, then move `top` down.
3. Traverse the right column from top to bottom, then move `right` left.
4. Traverse the bottom row from right to left (if still valid), then move `bottom` up.
5. Traverse the left column from bottom to top (if still valid), then move `left` right.
6. Repeat until boundaries cross.

3. When to Use It

- When the problem asks for "spiral order" or "clockwise spiral"
- When output must follow a specific circular/helical pattern
- When simulating a spiral path through a grid

Trigger phrases: "spiral order", "spiral matrix", "print in spiral", "clockwise spiral"

4. When Not to Use It

- When simple row-major or column-major traversal is sufficient
- When the matrix is very small (overhead of boundary tracking)
- When you need to modify the matrix (spiral traversal is primarily for reading)
- When the matrix is jagged or non-rectangular

5. Core Concepts

Concept	Explanation	Why It Matters
Boundary variables	<code>top</code> , <code>bottom</code> , <code>left</code> , <code>right</code> define the current layer	Controls traversal limits
Layer shrinking	After traversing a row/column, the corresponding boundary moves inward	Prevents revisiting cells
Direction order	Always <code>right</code> → <code>down</code> → <code>left</code> → <code>up</code>	This is the spiral sequence
Direction change	After traversing a full row or column, the direction rotates 90° clockwise	Ensures correct spiral path
Termination condition	When <code>top > bottom</code> or <code>left > right</code>	Loop ends when all layers are processed

6. Step-by-Step Algorithm

- 1. Initialize `top = 0, bottom = R-1, left = 0, right = C-1`.
- 2. While `top <= bottom` and `left <= right`: a. Traverse from `left` to `right` on row `top`. Increment `top`. b. Traverse from `top` to `bottom` on column `right`. Decrement `right`. c. If `top <= bottom`: traverse from `right` to `left` on row `bottom`. Decrement `bottom`. d. If `left <= right`: traverse from `bottom` to `top` on column `left`. Increment `left`.
- 3. Return the collected elements.

7. Dry Run

Matrix:

```
1  2  3  4
5  6  7  8
9 10 11 12
```

Step	Action	top	bottom	left	right	Output
Start	-	0	2	0	3	[]
1	Top row: 1,2,3,4	1	2	0	3	[1,2,3,4]
2	Right col: 8,12	1	2	0	2	[1,2,3,4,8,12]
3	Bottom row: 11,10,9	1	1	0	2	[1,2,3,4,8,12,11,10,9]
4	Left col: 5	1	1	1	2	[1,2,3,4,8,12,11,10,9,5]
5	Top row: 6,7	2	1	1	2	[1,2,3,4,8,12,11,10,9,5,6,7]
6	Right col: (none)	2	1	1	1	Stop (top > bottom)

Final: [1,2,3,4,8,12,11,10,9,5,6,7]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

vector<int> spiralOrder(vector<vector<int>>& matrix) {
    vector<int> result;
    if (matrix.empty()) return result;

    int R = matrix.size(), C = matrix[0].size();
    int top = 0, bottom = R - 1;
    int left = 0, right = C - 1;

    while (top <= bottom && left <= right) {
        // Traverse top row
        for (int j = left; j <= right; j++)
            result.push_back(matrix[top][j]);
        top++;
    }
```

```

        // Traverse right column
        for (int i = top; i <= bottom; i++)
            result.push_back(matrix[i][right]);
        right--;

        // Traverse bottom row (if valid)
        if (top <= bottom) {
            for (int j = right; j >= left; j--)
                result.push_back(matrix[bottom][j]);
            bottom--;
        }

        // Traverse left column (if valid)
        if (left <= right) {
            for (int i = bottom; i >= top; i--)
                result.push_back(matrix[i][left]);
            left++;
        }
    }
    return result;
}

int main() {
    vector<vector<int>> mat = {{1,2,3,4},{5,6,7,8},{9,10,11,12}};
    vector<int> res = spiralOrder(mat);
    for (int x : res) cout << x << " "; // 1 2 3 4 8 12 11 10 9 5 6 7
    return 0;
}

```

9. Python Implementation

```

from typing import List

def spiralOrder(matrix: List[List[int]]) -> List[int]:
    result = []
    if not matrix:
        return result

    R, C = len(matrix), len(matrix[0])
    top, bottom = 0, R - 1
    left, right = 0, C - 1

    while top <= bottom and left <= right:
        # Top row
        for j in range(left, right + 1):
            result.append(matrix[top][j])
        top += 1

        # Right column
        for i in range(top, bottom + 1):

```

```
        result.append(matrix[i][right])
    right -= 1

    # Bottom row
    if top <= bottom:
        for j in range(right, left - 1, -1):
            result.append(matrix[bottom][j])
        bottom -= 1

    # Left column
    if left <= right:
        for i in range(bottom, top - 1, -1):
            result.append(matrix[i][left])
        left += 1

    return result

mat = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]
print(spiralOrder(mat))
```

10. Code Explanation

- **Boundary initialization:** `top`, `bottom`, `left`, `right` define the current unvisited layer.
- **Top row traversal:** Move left to right on the top row. After finishing, shrink the top boundary.
- **Right column traversal:** Move top to bottom on the rightmost column. Shrink the right boundary.
- **Bottom row traversal:** Move right to left on the bottom row. Only if `top <= bottom` (still valid). Shrink bottom.
- **Left column traversal:** Move bottom to top on the leftmost column. Only if `left <= right`. Shrink left.
- **Guards:** The `if` checks after bottom and left traversals are essential for single-row/single-column matrices. Without them, you'd double-count cells.

11. Complexity Analysis

Operation	Time	Space
Full spiral traversal	$O(R \times C)$	$O(1)$ extra (excluding output)
Output storage	$O(R \times C)$	$O(R \times C)$

12. Common Patterns

Pattern	Identification	Approach	Example
Spiral matrix generation	"Generate matrix in spiral order"	Fill using same boundary technique	LeetCode 59
Spiral traversal from center	"Start from center, spiral outward"	Use direction array with expansion	Codeforces

Pattern	Identification	Approach	Example
Anti-clockwise spiral	"Counter-clockwise spiral"	Reverse direction order	LeetCode variations

13. Common Mistakes

- **Missing the `if` guards**: After traversing bottom row and left column, you must check if boundaries are still valid. Without this, a single-row matrix will print the same row twice.
- **Off-by-one in direction**: Using `<=` vs `<` incorrectly in the inner loops.
- **Not updating all boundaries**: Each of the four boundaries must be updated.
- **Assuming square matrix**: The algorithm works for any $R \times C$, but the guards are critical for non-square matrices.

14. Edge Cases

- Empty matrix: Return empty
- Single row ($R = 1$): No bottom or left traversal
- Single column ($C = 1$): No left traversal needed
- Single element: All four traversals won't trigger except the first
- Rectangular matrix ($R \neq C$): The inner layers may be a single row or column

15. Variations

Variation	Description	Importance
Spiral Matrix II	Generate $N \times N$ matrix filled in spiral order	High (LeetCode 59)
Anti-clockwise spiral	Reverse: left $\rightarrow$ down $\rightarrow$ right $\rightarrow$ up	Medium
Spiral from center	Start at center, spiral outward	Medium
Diagonal spiral	Traverse along diagonals in spiral pattern	Low

16. Related Algorithms/Data Structures

Related	Connection
Matrix rotation	Uses similar layer-by-layer approach
Snake traversal	Zigzag pattern, not spiral
Boundary traversal	Just the outer boundary (easier variant)

17. Practice Problems

Easy

- **Spiral Matrix** (LeetCode 54) — Return spiral order of matrix
- **Spiral Matrix II** (LeetCode 59) — Generate $N \times N$ spiral matrix

Medium

- **Spiral Matrix III** (LeetCode 885) — Spiral starting from center
- **Rotate Image** (LeetCode 48) — Layer-by-layer rotation

Hard

- **Valid Sudoku** (LeetCode 36) — Uses traversal patterns
- **Game of Life** (LeetCode 289) — In-place transformation with boundary traversal

18. Interview Explanation

"Spiral traversal uses a boundary-based approach. I maintain four pointers — top, bottom, left, right — that define the current layer. I traverse the top row, right column, bottom row, and left column of the current layer, then shrink the boundaries inward. The key trick is the `if` checks before the bottom and left traversals to handle single-row and single-column matrices correctly. The time complexity is $O(R \times C)$ since we visit each cell exactly once."

19. Revision Notes

- 4 boundaries: `top`, `bottom`, `left`, `right`
- Traverse: `right` → `down` → `left` → `up`
- After each traversal, shrink the boundary
- `if (top <= bottom)` before bottom row
- `if (left <= right)` before left column
- Single row/column: guards prevent double-counting
- Complexity: $O(R \times C)$ time, $O(1)$ extra space

20. Final Cheat Sheet

Aspect	Details
When to use	"Spiral order" problems
Main operations	Traverse boundary, shrink inward
Complexity	$O(R \times C)$ time, $O(1)$ extra space
Key code	<code>while(top<=bottom && left<=right)</code> with 4 inner loops
Edge cases	Empty, single row, single column, single element

3. ROTATE MATRIX

1. Overview

Rotate matrix means rotating a 2D array by 90 degrees (clockwise or counter-clockwise). The most common problem is rotating a square matrix by 90° clockwise in-place. This is a classic interview problem that tests understanding of index manipulation and in-place algorithms.

2. Intuition

Think of the matrix as a picture. Rotating it 90° clockwise means the top row becomes the rightmost column, the rightmost column becomes the bottom row, etc.

Analogy: Imagine a Rubik's cube face. When you rotate a face, each corner piece moves to the next corner position. This is exactly what happens with groups of 4 elements in the matrix.

Step-by-step reasoning:

- 1. A 90° rotation maps $(i, j) \rightarrow (j, R-1-i)$.
- 2. Instead of using extra space, we can rotate elements in groups of 4.
- 3. Process the matrix layer by layer (outer square, then inner square).
- 4. For each layer, process groups of 4 cells simultaneously.
- 5. Each group rotates 4 elements in a cycle.

Alternative approach: Transpose the matrix (swap rows with columns), then reverse each row. This gives a 90° clockwise rotation in two clean steps.

3. When to Use It

- When the problem asks to "rotate the image" or "rotate matrix"
- When you need to transform a matrix by 90° or 180°
- When working with image processing or pixel manipulation
- When the matrix orientation needs to change for a specific algorithm

Trigger phrases: "rotate image", "rotate matrix", "90 degree", "clockwise rotation", "in-place rotation"

4. When Not to Use It

- When the matrix is not square (use transpose + reshape instead)
- When you can use extra space (simpler to just create a new matrix)
- When the rotation angle is not a multiple of 90°
- When the matrix is extremely large (in-place rotation is cache-unfriendly)

5. Core Concepts

Concept	Explanation	Why It Matters
Transpose	Swap <code>mat[i][j]</code> with <code>mat[j][i]</code>	First step of the two-step approach
Row reversal	Reverse each row of the matrix	Second step of clockwise rotation
Layer-by-layer	Process outer perimeter, then inner perimeter	Enables in-place rotation
4-way swap	Swap 4 elements in a cycle: top→right→bottom→left→top	Core of the in-place algorithm
Index mapping	$(i, j) \rightarrow (j, R-1-i)$ for 90° clockwise	Mathematical understanding

6. Step-by-Step Algorithm

Approach 1: Transpose + Reverse (simple, uses extra knowledge)

1. Transpose the matrix: swap `mat[i][j]` with `mat[j][i]` for all $i < j$.
2. Reverse each row: for each row, reverse the elements.

Approach 2: 4-way swap (in-place, layer by layer)

1. Let R = number of rows.
2. For `layer = 0` to $R/2 - 1$:
 - Let `first = layer`, `last = R - 1 - layer`.
 - For $i = \text{first}$ to $\text{last} - 1$:
 - Store `top = mat[first][i]`
 - Move left $\rightarrow$ top: `mat[first][i] = mat[last - offset][first]`
 - Move bottom $\rightarrow$ left: `mat[last - offset][first] = mat[last][last - offset]`
 - Move right $\rightarrow$ bottom: `mat[last][last - offset] = mat[i][last]`
 - Move top $\rightarrow$ right: `mat[i][last] = top` (where `offset = i - first`)

7. Dry Run

Matrix:

```
1 2 3
4 5 6
7 8 9
```

Transpose + Reverse approach:

Step	Matrix
Original	1 2 3 / 4 5 6 / 7 8 9
After transpose	1 4 7 / 2 5 8 / 3 6 9
After row reversal	7 4 1 / 8 5 2 / 9 6 3

4-way swap approach:

Layer 0 (outer):

- `first=0, last=2`
- $i=0$: rotate $(0,0) \rightarrow (0,2) \rightarrow (2,2) \rightarrow (2,0) \rightarrow (0,0)$: $1 \rightarrow 3 \rightarrow 9 \rightarrow 7 \rightarrow 1$
- $i=1$: rotate $(0,1) \rightarrow (1,2) \rightarrow (2,1) \rightarrow (1,0) \rightarrow (0,1)$: $2 \rightarrow 6 \rightarrow 8 \rightarrow 4 \rightarrow 2$

Result:

```
7 4 1
8 5 2
9 6 3
```

8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

// Approach 1: Transpose + Reverse (90° clockwise)
void rotateTransposeReverse(vector<vector<int>>& matrix) {
    int R = matrix.size();
    // Transpose
    for (int i = 0; i < R; i++)
        for (int j = i + 1; j < R; j++)
            swap(matrix[i][j], matrix[j][i]);
    // Reverse each row
    for (int i = 0; i < R; i++)
        reverse(matrix[i].begin(), matrix[i].end());
}

// Approach 2: 4-way swap (in-place, layer by layer)
void rotate(vector<vector<int>>& matrix) {
    int R = matrix.size();
    for (int layer = 0; layer < R / 2; layer++) {
        int first = layer;
        int last = R - 1 - layer;
        for (int i = first; i < last; i++) {
            int offset = i - first;
            int top = matrix[first][i]; // save top

            // left -> top
            matrix[first][i] = matrix[last - offset][first];
            // bottom -> left
            matrix[last - offset][first] = matrix[last][last - offset];
            // right -> bottom
            matrix[last][last - offset] = matrix[i][last];
            // top -> right
            matrix[i][last] = top;
        }
    }
}

// 90° counter-clockwise rotation
void rotateCCW(vector<vector<int>>& matrix) {
    int R = matrix.size();
    // Transpose
    for (int i = 0; i < R; i++)
        for (int j = i + 1; j < R; j++)
            swap(matrix[i][j], matrix[j][i]);
    // Reverse each column
    for (int j = 0; j < R; j++)
        for (int i = 0; i < R / 2; i++)
            swap(matrix[i][j], matrix[R - 1 - i][j]);
}

```

```

int main() {
    vector<vector<int>> mat = {{1,2,3},{4,5,6},{7,8,9}};
    rotate(mat);
    for (auto& row : mat) {
        for (int x : row) cout << x << " ";
        cout << "\n";
    }
    // Output:
    // 7 4 1
    // 8 5 2
    // 9 6 3
    return 0;
}

```

9. Python Implementation

```

from typing import List

def rotate_clockwise(matrix: List[List[int]]) -> None:
    """Rotate 90° clockwise in-place."""
    R = len(matrix)
    # Transpose
    for i in range(R):
        for j in range(i + 1, R):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    # Reverse each row
    for i in range(R):
        matrix[i].reverse()

def rotate_ccw(matrix: List[List[int]]) -> None:
    """Rotate 90° counter-clockwise in-place."""
    R = len(matrix)
    # Transpose
    for i in range(R):
        for j in range(i + 1, R):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    # Reverse each column
    for j in range(R):
        for i in range(R // 2):
            matrix[i][j], matrix[R - 1 - i][j] = matrix[R - 1 - i][j], matrix[i]
[j]

# Example
mat = [[1,2,3],[4,5,6],[7,8,9]]
rotate_clockwise(mat)
print(mat) # [[7,4,1],[8,5,2],[9,6,3]]

```

10. Code Explanation

- **Transpose step:** Swapping `mat[i][j]` with `mat[j][i]` for all $i < j$ converts rows to columns. The $i < j$ condition ensures each pair is swapped exactly once.
- **Row reversal:** `reverse(mat[i].begin(), mat[i].end())` completes the 90° clockwise rotation.
- **4-way swap:** The corner elements are rotated in a cycle. `offset = i - first` tracks how far we are from the start of the layer, enabling correct mapping.
- **Layer loop:** We only need to process $R/2$ layers. For odd R , the center element stays in place.
- **Counter-clockwise:** Transpose + reverse each column instead of each row.

11. Complexity Analysis

Approach	Time	Space
Transpose + Reverse	$O(R^2)$	$O(1)$
4-way swap	$O(R^2)$	$O(1)$
Creating new matrix	$O(R^2)$	$O(R^2)$

12. Common Patterns

Pattern	Identification	Approach	Example
90° clockwise	"Rotate right"	Transpose + reverse row	LeetCode 48
90° counter-clockwise	"Rotate left"	Transpose + reverse column	Variation
180° rotation	"Upside down"	Reverse rows + reverse columns	Variation
Matrix reflection	"Mirror image"	Reverse rows (horizontal) or columns (vertical)	Easy

13. Common Mistakes

- **Forgetting $i < j$ in transpose:** If you swap all pairs, you'll swap back and undo the transpose.
- **Using $O(R^2)$ extra space:** Interviewers expect in-place rotation.
- **Assuming square matrix:** The standard rotation formula is only for square matrices.
- **Wrong index mapping:** The 4-way swap requires careful offset tracking.
- **Not handling the center element:** For odd-sized matrices, the center stays unchanged.

14. Edge Cases

- Empty matrix: No operation
- 1×1 matrix: No operation
- 2×2 matrix: Single layer, 1 group of 4
- Odd-sized matrix: Center element stays in place
- Large matrix: Performance of cache-friendly approach matters

15. Variations

Variation	Description	Importance
Rotate non-square	Use extra space, or transpose + reshape	Medium

Variation	Description	Importance
Rotate 180°	Reverse rows + reverse columns, or two 90° rotations	Low
Rotate by 90° counter-clockwise	Transpose + reverse columns	Medium
Rotate k times	$k \% 4$ determines which direction	Medium

16. Related Algorithms/Data Structures

Related	Connection
Matrix transpose	Core building block of rotation
Spiral traversal	Also uses layer-by-layer approach
Image processing	Rotation is a fundamental image operation

17. Practice Problems

Easy

- **Rotate Image** (LeetCode 48) — In-place 90° rotation of $N \times N$ matrix
- **Transpose Matrix** (LeetCode 867) — Simple transpose (non-square allowed)

Medium

- **Rotate Array** (LeetCode 189) — 1D rotation (related concept)
- **Battleships in a Board** (LeetCode 419) — Uses traversal patterns

Hard

- **Image Overlap** (LeetCode 835) — Combines rotation with convolution
- **Rotate String** (LeetCode 796) — String rotation (different domain, same pattern)

18. Interview Explanation

"To rotate a square matrix by 90° clockwise in-place, I use two steps: transpose (swap rows with columns) and then reverse each row. The transpose step swaps `mat[i][j]` with `mat[j][i]` for all $i < j$, and then reversing each row completes the rotation. For counter-clockwise, I reverse each column instead. This approach is $O(R^2)$ time and $O(1)$ extra space. Alternatively, I can use a 4-way swap approach layer by layer, where each group of 4 elements is rotated in a cycle."

19. Revision Notes

- 90° clockwise: Transpose + Reverse each row
- 90° counter-clockwise: Transpose + Reverse each column
- 180°: Reverse rows + Reverse columns (or two 90° rotations)
- Transpose: swap `mat[i][j]` with `mat[j][i]` for $i < j$
- In-place: $O(R^2)$ time, $O(1)$ space
- Only works for square matrices

- k rotations: $k \% 4$ determines direction

20. Final Cheat Sheet

Aspect	Details
When to use	"Rotate image", "90 degree" problems
Main operations	Transpose + reverse (or 4-way swap)
Complexity	$O(R^2)$ time, $O(1)$ space
Key code	<code>transpose</code> then <code>reverse each row</code>
Edge cases	Non-square, empty, 1×1 , odd-sized

4. SEARCH IN SORTED MATRIX

1. Overview

Searching in a sorted matrix involves finding a target value in a 2D matrix where rows and/or columns are sorted. The most common variant is a matrix where each row is sorted left-to-right and each column is sorted top-to-bottom (the "Young tableau" property). The optimal algorithm is $O(R + C)$ time.

2. Intuition

Analogy: Imagine you're standing at the top-right corner of a matrix. If the target is larger than the current element, you know it must be below (since all elements below are larger in that column). If the target is smaller, it must be to the left (since all elements to the left are smaller in that row). You can eliminate one row or one column at each step.

Why it works: The matrix has the property that rows are sorted left-to-right and columns are sorted top-to-bottom. This means:

- At any cell, all elements below it are larger.
- All elements to the left are smaller.
- By starting at the top-right (or bottom-left), you can eliminate one direction at each step.

Step-by-step reasoning:

1. Start at `top-right` corner: `r = 0, c = C-1`.
2. While `r < R` and `c >= 0`:
 - If `mat[r][c] == target`: return true.
 - If `mat[r][c] > target`: move left (`c--`).
 - If `mat[r][c] < target`: move down (`r++`).
3. If we exit the loop, target is not found.

3. When to Use It

- When the matrix rows are sorted AND columns are sorted

- When the matrix is a "Young tableau" (row-sorted, column-sorted)
- When you need $O(R + C)$ search instead of $O(R \log C)$ binary search
- When the matrix is large and you need an efficient search

Trigger phrases: "search in sorted matrix", "search 2D matrix", "row and column sorted", "young tableau"

4. When Not to Use It

- When only rows are sorted (use binary search on each row: $O(R \log C)$)
- When the matrix is small (linear scan is simpler)
- When the matrix is not sorted (use hash set)
- When you need to find multiple elements (use preprocessing)
- When the matrix is guaranteed to be fully sorted (binary search on flattened array: $O(\log(R \times C))$)

5. Core Concepts

Concept	Explanation	Why It Matters
Top-right start	Starting position for the elimination approach	Key to $O(R+C)$ algorithm
Bottom-left start	Alternative starting position	Equivalent to top-right
Elimination	At each step, eliminate one row or column	Ensures linear time
Row-sorted property	Each row is sorted left-to-right	Enables left/right decisions
Column-sorted property	Each column is sorted top-to-bottom	Enables up/down decisions

6. Step-by-Step Algorithm

1. Get $R = \text{rows}$, $C = \text{columns}$.
2. Initialize $r = 0$, $c = C - 1$ (top-right corner).
3. Loop while $r < R$ and $c \geq 0$:
 - If $\text{matrix}[r][c] == \text{target}$: return `true`.
 - If $\text{matrix}[r][c] > \text{target}$: $c--$ (move left — current element is too large).
 - Else: $r++$ (move down — current element is too small).
4. Return `false` (target not found).

7. Dry Run

Matrix:

1	4	7	11
2	5	8	12
3	6	9	16
10	13	14	17

Search for `target = 9`:

Step	r	c	mat[r][c]	Action
1	0	3	11	11 > 9, move left
2	0	2	7	7 < 9, move down
3	1	2	8	8 < 9, move down
4	2	2	9	Found!

Search for `target = 15`:

Step	r	c	mat[r][c]	Action
1	0	3	11	11 < 15, move down
2	1	3	12	12 < 15, move down
3	2	3	16	16 > 15, move left
4	2	2	9	9 < 15, move down
5	3	2	14	14 < 15, move down
6	4	2	—	Out of bounds, not found

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Search in row-wise AND column-wise sorted matrix
// Time: O(R + C), Space: O(1)
bool searchMatrix(vector<vector<int>>& matrix, int target) {
    if (matrix.empty()) return false;
    int R = matrix.size(), C = matrix[0].size();
    int r = 0, c = C - 1; // start at top-right

    while (r < R && c >= 0) {
        if (matrix[r][c] == target) return true;
        if (matrix[r][c] > target)
            c--; // move left
        else
            r++; // move down
    }
    return false;
}

// Binary search approach (when only rows are sorted)
// Time: O(R log C), Space: O(1)
bool searchMatrixBS(vector<vector<int>>& matrix, int target) {
    if (matrix.empty()) return false;
    int R = matrix.size(), C = matrix[0].size();
    int lo = 0, hi = R * C - 1;
```

```

// Treat as flattened sorted array
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    int val = matrix[mid / C][mid % C];
    if (val == target) return true;
    if (val < target) lo = mid + 1;
    else hi = mid - 1;
}
return false;
}

int main() {
    vector<vector<int>> mat = {
        {1, 4, 7, 11},
        {2, 5, 8, 12},
        {3, 6, 9, 16},
        {10, 13, 14, 17}
    };
    cout << searchMatrix(mat, 9) << "\n"; // 1 (true)
    cout << searchMatrix(mat, 15) << "\n"; // 0 (false)
    return 0;
}

```

9. Python Implementation

```

from typing import List

def searchMatrix(matrix: List[List[int]], target: int) -> bool:
    if not matrix or not matrix[0]:
        return False
    R, C = len(matrix), len(matrix[0])
    r, c = 0, C - 1 # top-right

    while r < R and c >= 0:
        if matrix[r][c] == target:
            return True
        if matrix[r][c] > target:
            c -= 1 # move left
        else:
            r += 1 # move down
    return False

# Binary search version (fully sorted matrix flattened)
def searchMatrixBS(matrix: List[List[int]], target: int) -> bool:
    if not matrix or not matrix[0]:
        return False
    R, C = len(matrix), len(matrix[0])
    lo, hi = 0, R * C - 1
    while lo <= hi:
        mid = (lo + hi) // 2

```

```

        val = matrix[mid // C][mid % C]
        if val == target:
            return True
        if val < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return False

mat = [[1,4,7,11],[2,5,8,12],[3,6,9,16],[10,13,14,17]]
print(searchMatrix(mat, 9))    # True
print(searchMatrix(mat, 15))   # False

```

10. Code Explanation

- **Top-right start:** The starting position (0, C-1) is key. At this position, moving left decreases the value (since rows are sorted) and moving down increases the value (since columns are sorted).
- **Elimination logic:** If `matrix[r][c] > target`, all elements below this cell are even larger, so we move left. If `matrix[r][c] < target`, all elements to the left are even smaller, so we move down.
- **Binary search variant:** For the "fully sorted" matrix (each row sorted and the last element of row $i \leq$ first element of row $i+1$), we can treat the matrix as a flattened sorted array and use standard binary search.
- **Edge case:** Empty matrix returns false immediately.

11. Complexity Analysis

Variant	Time	Space
Elimination (top-right)	$O(R + C)$	$O(1)$
Binary search on flattened	$O(\log(R \times C))$	$O(1)$
Binary search per row	$O(R \log C)$	$O(1)$
Linear scan	$O(R \times C)$	$O(1)$

12. Common Patterns

Pattern	Identification	Approach	Example
Row & column sorted	"Each row and column sorted"	Top-right elimination	LeetCode 240
Fully sorted matrix	"Each row sorted, last $\leq$ first of next"	Binary search on flattened	LeetCode 74
Count negative numbers	"Count negatives in sorted matrix"	Elimination from bottom-left	LeetCode 1351
Kth smallest element	"Kth smallest in sorted matrix"	Binary search on value range	LeetCode 378

13. Common Mistakes

- **Using top-left instead of top-right:** Starting at top-left gives no elimination power (both directions are larger).
- **Forgetting the column-sorted property:** The algorithm only works if both rows and columns are sorted.
- **Using binary search on partially sorted matrix:** Binary search on flattened array only works when row transitions are monotonic.
- **Off-by-one in boundary checks:** $r < R$ and $c \geq 0$ (not $c > 0$).

14. Edge Cases

- Empty matrix: Return false
- Single row: Works fine (moves left only)
- Single column: Works fine (moves down only)
- Target smaller than all elements: Ends at $c = -1$
- Target larger than all elements: Ends at $r = R$
- Duplicate values: Still works (first match returns)

15. Variations

Variation	Description	Importance
Search in fully sorted matrix	Row transitions are monotonic, binary search on flattened	High (LeetCode 74)
Count negatives in sorted matrix	Count elements < 0 using elimination	High (LeetCode 1351)
Kth smallest in sorted matrix	Binary search on value range with counting	High (LeetCode 378)
Search with duplicates	Standard elimination still works	Medium

16. Related Algorithms/Data Structures

Related	Connection
Binary search	Used for fully sorted variant
Two pointers	Similar elimination technique
Divide and conquer	More complex matrix search (quadrant elimination)

17. Practice Problems

Easy

- **Search a 2D Matrix** (LeetCode 74) — Fully sorted matrix, binary search
- **Count Negative Numbers in Sorted Matrix** (LeetCode 1351) — Count negatives

Medium

- **Search a 2D Matrix II** (LeetCode 240) — Row & column sorted, elimination
- **Kth Smallest Element in a Sorted Matrix** (LeetCode 378) — Binary search on value

Hard

- **Median of a Row-Wise Sorted Matrix** (GFG) — Binary search on value range
- **Maximum Sum Submatrix** (LeetCode 363) — Uses prefix sum + binary search

18. Interview Explanation

"For searching in a row-wise and column-wise sorted matrix, I start at the top-right corner. At each step, if the current element equals the target, I return true. If it's greater than the target, I move left; if smaller, I move down. This works because at top-right, moving left decreases the value and moving down increases it. The time complexity is $O(R + C)$. For the variant where the matrix is fully sorted (each row and the last element $\leq$ first of next), I can use binary search on the flattened array for $O(\log R \times C)$ time."

19. Revision Notes

- Start at top-right: $r = 0, c = C - 1$
- If $mat[r][c] > target$: move left
- If $mat[r][c] < target$: move down
- If equal: return true
- $O(R + C)$ time, $O(1)$ space
- For fully sorted matrix: binary search on flattened array
- Works only when both rows and columns are sorted

20. Final Cheat Sheet

Aspect	Details
When to use	Row & column sorted matrix search
Main operations	Move left or down from top-right
Complexity	$O(R + C)$ time, $O(1)$ space
Key code	<pre>while(r<R && c>=0) { if(mat[r][c]==target) return; else if(mat[r][c]>target) c--; else r++; }</pre>
Edge cases	Empty, single row/col, target out of range

5. FLOOD FILL

1. Overview

Flood fill is an algorithm that determines the area connected to a given node in a multi-dimensional array. It "fills" all connected cells of the same color/value with a new color/value. It is the algorithm behind the paint bucket tool in graphics editors.

2. Intuition

Analogy: Imagine pouring water on a tile of a colored floor. The water spreads to all adjacent tiles of the same color, but stops when it hits a different color. This is exactly how the paint bucket tool works.

Why it works: The algorithm is essentially a graph traversal (BFS or DFS) on a grid where edges exist between cells of the same color. Starting from the source cell, we explore all reachable cells that share the original color.

Step-by-step reasoning:

1. Start at the given cell (**sr**, **sc**).
2. If the cell's color is already the target color, return (no work needed).
3. Save the original color.
4. Change the current cell to the target color.
5. For each 4-directional neighbor:
 - If the neighbor is within bounds and has the original color, recursively flood fill it.

3. When to Use It

- When you need to change all connected cells of the same value
- When implementing a paint bucket tool
- When finding connected components (islands) in a grid
- When you need to "fill" a region bounded by a different color
- When simulating the spread of something through a grid

Trigger phrases: "flood fill", "paint bucket", "fill connected", "spread to adjacent", "same color region"

4. When Not to Use It

- When you only need to find connected components (use DFS/BFS for counting instead)
- When the grid is very large and recursion depth is a concern (use BFS/iterative DFS)
- When you need to fill a region bounded by a specific color (use boundary fill)
- When the region is already the target color (just return early)

5. Core Concepts

Concept	Explanation	Why It Matters
Source cell	The starting cell for the fill	Determines which region is filled
Original color	The color/value of the source cell before filling	Used to identify connected cells
Target color	The new color to fill with	The replacement value

Concept	Explanation	Why It Matters
4-directional connectivity	Up, down, left, right neighbors	Standard for flood fill (8-directional is also possible)
Recursion/stack	DFS approach using recursion or explicit stack	Controls exploration order
Queue	BFS approach using queue	Level-by-level expansion

6. Step-by-Step Algorithm

DFS (Recursive) approach:

- 1. If `image[sr][sc] == newColor`, return (already filled).
- 2. Save `originalColor = image[sr][sc]`.
- 3. Call `dfs(sr, sc)`:
 - Set `image[r][c] = newColor`.
 - For each 4-directional neighbor `(nr, nc)`:
 - If `nr, nc` is valid AND `image[nr][nc] == originalColor`:
 - Call `dfs(nr, nc)`.

BFS (Iterative) approach:

- 1. If `image[sr][sc] == newColor`, return.
- 2. Save `originalColor = image[sr][sc]`.
- 3. Create a queue and push `(sr, sc)`.
- 4. While queue is not empty:
 - Pop `(r, c)`.
 - Set `image[r][c] = newColor`.
 - For each neighbor:
 - If valid AND `image[nr][nc] == originalColor`:
 - Push `(nr, nc)`.

7. Dry Run

Image:

```
1 1 1
1 1 0
1 0 1
```

Start: `(1, 1)`, newColor: 2

Step	Queue/Stack	Grid State
Start	[(1,1)]	1 1 1 / 1 1 0 / 1 0 1
1	Pop (1,1), fill → 2, push (0,1), (1,0), (1,2), (2,1)	1 1 1 / 1 2 0 / 1 0 1

Step	Queue/Stack	Grid State
2	(0,1) fill → 2, push (0,0), (0,2)	1 2 1 / 1 2 0 / 1 0 1
3	(1,0) fill → 2, push (2,0)	1 2 1 / 2 2 0 / 1 0 1
4	(1,2) → color is 0, skip	1 2 1 / 2 2 0 / 1 0 1
5	(2,1) → color is 0, skip	1 2 1 / 2 2 0 / 1 0 1
6	(0,0) fill → 2, push neighbors...	2 2 1 / 2 2 0 / 1 0 1
7	(0,2) → color is 1, fill → 2	2 2 2 / 2 2 0 / 1 0 1
8	(2,0) → color is 1, fill → 2	2 2 2 / 2 2 0 / 2 0 1
...	Continue until queue empty	

Final:

```
2 2 2
2 2 0
2 0 1
```

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// DFS Recursive approach
class Solution {
    int R, C, originalColor;
    vector<vector<int>>* img;

    void dfs(int r, int c, int newColor) {
        if (r < 0 || r >= R || c < 0 || c >= C) return;
        if ((*img)[r][c] != originalColor) return;

        (*img)[r][c] = newColor;

        dfs(r - 1, c, newColor); // up
        dfs(r + 1, c, newColor); // down
        dfs(r, c - 1, newColor); // left
        dfs(r, c + 1, newColor); // right
    }

public:
    vector<vector<int>> floodFill(vector<vector<int>>& image, int sr, int sc, int
newColor) {
        if (image.empty()) return image;
        originalColor = image[sr][sc];
        if (originalColor == newColor) return image; // no work needed
    }
};
```

```

        R = image.size();
        C = image[0].size();
        img = &image;

        dfs(sr, sc, newColor);
        return image;
    }
};

// BFS Iterative approach
vector<vector<int>>> floodFillBFS(vector<vector<int>>& image, int sr, int sc, int
newColor) {
    if (image.empty()) return image;
    int originalColor = image[sr][sc];
    if (originalColor == newColor) return image;

    int R = image.size(), C = image[0].size();
    queue<pair<int,int>> q;
    q.push({sr, sc});

    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};

    while (!q.empty()) {
        auto [r, c] = q.front(); q.pop();

        if (r < 0 || r >= R || c < 0 || c >= C) continue;
        if (image[r][c] != originalColor) continue;

        image[r][c] = newColor;

        for (int k = 0; k < 4; k++)
            q.push({r + dr[k], c + dc[k]});
    }
    return image;
}

int main() {
    vector<vector<int>>> img = {{1,1,1},{1,1,0},{1,0,1}};
    Solution sol;
    auto res = sol.floodFill(img, 1, 1, 2);
    for (auto& row : res) {
        for (int x : row) cout << x << " ";
        cout << "\n";
    }
    // Output:
    // 2 2 2
    // 2 2 0
    // 2 0 1
    return 0;
}

```

9. Python Implementation

```

from typing import List
from collections import deque

# DFS Recursive
def floodFill(image: List[List[int]], sr: int, sc: int, newColor: int) -> List[List[int]]:
    if not image or not image[0]:
        return image

    R, C = len(image), len(image[0])
    original = image[sr][sc]
    if original == newColor:
        return image

    def dfs(r, c):
        if r < 0 or r >= R or c < 0 or c >= C:
            return
        if image[r][c] != original:
            return
        image[r][c] = newColor
        dfs(r - 1, c)
        dfs(r + 1, c)
        dfs(r, c - 1)
        dfs(r, c + 1)

    dfs(sr, sc)
    return image

# BFS Iterative
def floodFillBFS(image: List[List[int]], sr: int, sc: int, newColor: int) -> List[List[int]]:
    if not image or not image[0]:
        return image

    R, C = len(image), len(image[0])
    original = image[sr][sc]
    if original == newColor:
        return image

    q = deque([(sr, sc)])
    dirs = [(-1,0), (1,0), (0,-1), (0,1)]

    while q:
        r, c = q.popleft()
        if r < 0 or r >= R or c < 0 or c >= C:
            continue
        if image[r][c] != original:
            continue
        image[r][c] = newColor
        for dr, dc in dirs:

```

```

        q.append((r + dr, c + dc))

    return image

# Example
img = [[1,1,1],[1,1,0],[1,0,1]]
print(floodFill(img, 1, 1, 2))

```

10. Code Explanation

- **DFS approach:** Simple recursive implementation. The base cases check bounds and original color. The function marks the current cell and recurses to all 4 neighbors.
- **BFS approach:** Uses a queue. Early exit if `originalColor == newColor` prevents infinite loops. The queue processes cells level by level.
- **Early return:** If `originalColor == newColor`, we return immediately. Without this check, the algorithm would check the same cells repeatedly (though it would still work with the BFS approach since we check color before filling).
- **Class-based encapsulation:** The `Solution` class stores state to avoid passing it in every recursive call.

11. Complexity Analysis

Approach	Time	Space
DFS (recursive)	$O(R \times C)$	$O(R \times C)$ worst-case recursion stack
BFS (iterative)	$O(R \times C)$	$O(R \times C)$ queue size
Both	$O(N)$ where N = number of cells in region	$O(N)$ for stack/queue

12. Common Patterns

Pattern	Identification	Approach	Example
Paint bucket	"Fill connected region"	Standard flood fill	LeetCode 733
Boundary fill	"Fill region bounded by color"	Flood fill with boundary check	Computer graphics
Island counting	"Count connected components"	Flood fill each unvisited cell	LeetCode 200
Replace all occurrences	"Replace all X with Y"	Flood fill from multiple sources	GFG

13. Common Mistakes

- **Forgetting the early return:** If `originalColor == newColor`, the algorithm may loop infinitely (DFS stack overflow) or waste time.
- **Using wrong connectivity:** Default is 4-directional, but some problems need 8-directional (including diagonals).

- **Not checking bounds:** Out-of-bounds access causes runtime errors.
- **Modifying the original color while iterating:** In BFS, you must check the original color before setting the new color. If you check after pushing, you might push duplicates.
- **Stack overflow:** For large grids, DFS recursion may overflow the stack. Use BFS or iterative DFS.

14. Edge Cases

- Empty image: Return empty
- 1×1 image: Single cell, fill it
- Already filled: `originalColor == newColor`, return early
- All cells same color: Entire grid gets filled
- Disconnected regions: Only the connected region is filled
- Grid with single row or column: Boundary checks handle it

15. Variations

Variation	Description	Importance
8-directional flood fill	Include diagonals as neighbors	Medium
Boundary fill	Fill until a boundary color is hit	Medium
Scanline fill	Optimized version for large regions	Low
Multiple source fill	Start from multiple cells simultaneously	Medium

16. Related Algorithms/Data Structures

Related	Connection
DFS / BFS on grid	Flood fill IS a graph traversal on grid
Number of Islands	Uses flood fill to mark visited islands
Connected components	Flood fill is how components are labeled
Union-Find (DSU)	Alternative for connected components on grid

17. Practice Problems

Easy

- **Flood Fill** (LeetCode 733) — Standard flood fill problem
- **Island Perimeter** (LeetCode 463) — Count perimeter of island

Medium

- **Number of Islands** (LeetCode 200) — Count connected components
- **Max Area of Island** (LeetCode 695) — Find largest connected component

Hard

- **Making A Large Island** (LeetCode 827) — Change one 0 to 1 to maximize island
- **Pacific Atlantic Water Flow** (LeetCode 417) — Multi-source flood fill

18. Interview Explanation

"Flood fill is a graph traversal algorithm on a grid. Starting from a source cell, we visit all connected cells of the same color and change them to a new color. I implement it using DFS or BFS. The key edge case is when the original color equals the new color — in that case, we return early to avoid unnecessary work. The time complexity is $O(N)$ where N is the number of cells in the filled region."

19. Revision Notes

- DFS: recursive, check bounds + color, mark, recurse to neighbors
- BFS: queue-based, level-order expansion
- Early return if `original == newColor`
- 4-directional by default (check problem for 8-directional)
- Time: $O(N)$, Space: $O(N)$ (stack/queue)
- Same as DFS/BFS on grid

20. Final Cheat Sheet

Aspect	Details
When to use	Fill connected region of same color
Main operations	DFS or BFS from source, check color before filling
Complexity	$O(N)$ time, $O(N)$ space
Key code	<code>if(color!=original) return; color=newColor; recurse/queue neighbors</code>
Edge cases	Empty, 1×1 , <code>original == newColor</code> , all same color

6. NUMBER OF ISLANDS

1. Overview

The "Number of Islands" problem asks to count the number of connected components of '1's (land) in a 2D grid of '1's and '0's (water). It is a classic graph traversal problem that tests understanding of DFS/BFS on a grid.

2. Intuition

Analogy: Imagine an aerial view of an archipelago. Each island is a connected landmass surrounded by water. Two land cells are connected if they are adjacent horizontally or vertically. We need to count how many distinct islands exist.

Why it works: The grid is a graph where each cell is a node and edges exist between adjacent land cells. Counting islands is equivalent to counting connected components of this graph. We traverse each component

fully and mark it as visited, then move to the next unvisited land cell.

Step-by-step reasoning:

1. Scan the grid cell by cell.
2. When we find a '1' that hasn't been visited, we increment the island count.
3. From that cell, perform DFS/BFS to visit all connected '1's (mark them as visited).
4. Continue scanning for the next unvisited '1'.
5. Each time we start a new traversal, we've found a new island.

3. When to Use It

- When counting connected components of a specific value in a grid
- When finding distinct regions/clusters in a 2D array
- When the problem asks for "number of islands", "connected components", "clusters"
- As a building block for more complex grid problems

Trigger phrases: "number of islands", "count islands", "connected components in grid", "distinct regions"

4. When Not to Use It

- When the grid is extremely large (use Union-Find/DSU for better performance)
- When you need to count islands of multiple types (use separate counters)
- When the grid changes dynamically (use DSU with updates)
- When you need to find the maximum island area (same algorithm, track size)

5. Core Concepts

Concept	Explanation	Why It Matters
Visited marking	Mark land cells as visited to avoid recounting	Can be done in-place (modify grid) or with a separate visited array
Component definition	4-directionally connected '1's form one island	8-directional is also possible (check problem)
Traversal trigger	An unvisited '1' starts a new island count	Each trigger increments the counter
Sink technique	Change '1' to '0' to mark visited (in-place)	Saves space, modifies input
DFS/BFS choice	Either works for traversal	DFS is simpler, BFS avoids stack overflow

6. Step-by-Step Algorithm

DFS approach:

1. Initialize **count** = 0.
2. For each cell (**r**, **c**) in the grid:
 - If **grid[r][c] == '1'**:

- Increment `count`.
 - Call `dfs(r, c)` to sink the entire island.
3. Return `count`.

`dfs(r, c)`:

1. If out of bounds or `grid[r][c] != '1'`, return.
2. Set `grid[r][c] = '0'` (sink it).
3. Recursively call `dfs` on all 4 neighbors.

7. Dry Run

Grid:

```
1 1 0 0 0
1 1 0 0 0
0 0 1 0 0
0 0 0 1 1
```

Step	Cell	grid[r][c]	Action	Count
1	(0,0)	1	Start DFS, sink island at (0,0)	1
2	(0,0)	0	Sink (0,0), visit (0,1), (1,0)	1
3	(0,1)	0	Sink (0,1)	1
4	(1,0)	0	Sink (1,0)	1
5	(1,1)	0	Sink (1,1)	1
6	(0,2)	0	Skip	1
7	(2,2)	1	Start DFS, sink island at (2,2)	2
8	(2,2)	0	Sink (2,2)	2
9	(3,3)	1	Start DFS, sink island at (3,3)	3
10	(3,3)	0	Sink (3,3), visit (3,4)	3
11	(3,4)	0	Sink (3,4)	3

Final count: 3

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
    int R, C;
```

```

void dfs(vector<vector<char>>& grid, int r, int c) {
    if (r < 0 || r >= R || c < 0 || c >= C) return;
    if (grid[r][c] != '1') return;

    grid[r][c] = '0'; // sink the cell

    dfs(grid, r - 1, c); // up
    dfs(grid, r + 1, c); // down
    dfs(grid, r, c - 1); // left
    dfs(grid, r, c + 1); // right
}

public:
int numIslands(vector<vector<char>>& grid) {
    if (grid.empty()) return 0;
    R = grid.size();
    C = grid[0].size();
    int count = 0;

    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            if (grid[i][j] == '1') {
                count++;
                dfs(grid, i, j); // sink the entire island
            }
        }
    }
    return count;
}

};

// BFS approach
int numIslandsBFS(vector<vector<char>>& grid) {
    if (grid.empty()) return 0;
    int R = grid.size(), C = grid[0].size();
    int count = 0;
    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};

    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            if (grid[i][j] == '1') {
                count++;
                queue<pair<int,int>> q;
                q.push({i, j});
                grid[i][j] = '0';

                while (!q.empty()) {
                    auto [r, c] = q.front(); q.pop();
                    for (int k = 0; k < 4; k++) {
                        int nr = r + dr[k], nc = c + dc[k];
                        if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr][nc]
== '1') {

```

```

        grid[nr][nc] = '0';
        q.push({nr, nc});
    }
}
}
}
}
}
return count;
}

int main() {
    vector<vector<char>> grid = {
        {'1','1','0','0','0'},
        {'1','1','0','0','0'},
        {'0','0','1','0','0'},
        {'0','0','0','1','1'}
    };
    Solution sol;
    cout << sol.numIslands(grid) << "\n"; // 3
    return 0;
}

```

9. Python Implementation

```

from typing import List
from collections import deque

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        if not grid or not grid[0]:
            return 0

        R, C = len(grid), len(grid[0])
        count = 0

        def dfs(r, c):
            if r < 0 or r >= R or c < 0 or c >= C:
                return
            if grid[r][c] != '1':
                return
            grid[r][c] = '0' # sink
            dfs(r - 1, c)
            dfs(r + 1, c)
            dfs(r, c - 1)
            dfs(r, c + 1)

        for i in range(R):
            for j in range(C):
                if grid[i][j] == '1':
                    count += 1
                    dfs(i, j)

```

```

        dfs(i, j)

    return count

# BFS version
def numIslandsBFS(grid: List[List[str]]) -> int:
    if not grid or not grid[0]:
        return 0

    R, C = len(grid), len(grid[0])
    count = 0
    dirs = [(-1,0), (1,0), (0,-1), (0,1)]

    for i in range(R):
        for j in range(C):
            if grid[i][j] == '1':
                count += 1
                q = deque([(i, j)])
                grid[i][j] = '0'

                while q:
                    r, c = q.popleft()
                    for dr, dc in dirs:
                        nr, nc = r + dr, c + dc
                        if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] == '1':
                            grid[nr][nc] = '0'
                            q.append((nr, nc))

    return count

grid = [
    ["1","1","0","0","0"],
    ["1","1","0","0","0"],
    ["0","0","1","0","0"],
    ["0","0","0","1","1"]
]
print(Solution().numIslands(grid)) # 3

```

10. Code Explanation

- **DFS function:** The recursive `dfs` function sinks the current cell (sets it to '0') and then recurses to all 4 neighbors. The base cases check bounds and whether the cell is land.
- **Main loop:** Iterates through every cell. When it finds a '1', it increments the count and starts a DFS to sink the entire connected component.
- **Sink technique:** By modifying the grid in-place, we avoid using extra memory for a `visited` array. This is acceptable for interviews but be aware it modifies the input.
- **BFS approach:** Uses a queue instead of recursion. The cell is marked as visited ('0') when pushed to the queue to avoid duplicate processing.

11. Complexity Analysis

Approach	Time	Space
DFS (recursive)	$O(R \times C)$	$O(R \times C)$ worst-case recursion stack
BFS (iterative)	$O(R \times C)$	$O(\min(R, C))$ queue size
Both	$O(N)$ where N = total cells	Depends on approach

12. Common Patterns

Pattern	Identification	Approach	Example
Standard island count	"Count islands"	DFS/BFS + sink	LeetCode 200
Max area of island	"Largest island"	Track size during DFS	LeetCode 695
Number of distinct islands	"Count distinct shapes"	Store shape signature	LeetCode 694
Island perimeter	"Perimeter of island"	Count water neighbors	LeetCode 463
Making a large island	"Change one 0 to 1 to maximize"	Component labeling + DSU	LeetCode 827

13. Common Mistakes

- **Not handling empty grid:** Check `grid.empty()` or `grid[0].empty()`.
- **Forgetting to mark visited:** Without sinking, the algorithm will loop infinitely.
- **Using wrong character:** The grid contains characters '0'/'1', not integers.
- **8-directional vs 4-directional:** Most problems use 4-directional. Check carefully.
- **Stack overflow:** For very large grids with DFS recursion, use BFS instead.
- **Modifying input without permission:** Some interviewers prefer a separate `visited` array.

14. Edge Cases

- Empty grid: Return 0
- Grid with no land (all '0'): Return 0
- Grid with all land (all '1'): Return 1
- Single row: Works fine
- Single column: Works fine
- 1×1 grid with '1': Return 1
- 1×1 grid with '0': Return 0

15. Variations

Variation	Description	Importance
Max Area of Island	Return size of largest island	High (LeetCode 695)

Variation	Description	Importance
Number of Distinct Islands	Count islands with unique shapes	High (LeetCode 694)
Island Perimeter	Calculate perimeter of all islands	Medium (LeetCode 463)
Making A Large Island	Flip one 0 to 1 to maximize island size	Hard (LeetCode 827)
Number of Closed Islands	Count islands completely surrounded by water	Medium (LeetCode 1254)

16. Related Algorithms/Data Structures

Related	Connection
Flood Fill	Same traversal, different purpose
BFS/DFS on grid	Core traversal mechanism
Union-Find (DSU)	Alternative for counting components
Connected Components	General graph problem, grid is a special case

17. Practice Problems

Easy

- **Number of Islands** (LeetCode 200) — Standard island counting
- **Island Perimeter** (LeetCode 463) — Count edges of island

Medium

- **Max Area of Island** (LeetCode 695) — Find largest island
- **Number of Distinct Islands** (LeetCode 694, LeetCode 305) — Unique shapes
- **Number of Closed Islands** (LeetCode 1254) — Fully surrounded islands

Hard

- **Making A Large Island** (LeetCode 827) — Flip one 0 to 1 for max island
- **Number of Islands II** (LeetCode 305) — Dynamic island addition with DSU

18. Interview Explanation

"To count the number of islands, I scan the grid and whenever I find a '1' that hasn't been visited, I increment the count and perform a DFS (or BFS) to mark the entire connected component as visited. I use the 'sink' technique where I change '1' to '0' to mark visited cells, which uses $O(1)$ extra space. The time complexity is $O(R \times C)$ since each cell is visited once. The main edge cases are empty grids and grids with no land."

19. Revision Notes

- Scan grid, on '1' $\rightarrow$ count++, DFS to sink entire island
- Sink: set `grid[r][c] = '0'`

- 4-directional neighbors
- $O(R \times C)$ time, $O(1)$ space (with sink) or $O(R \times C)$ (with visited array)
- BFS avoids stack overflow for large grids
- Character grid: '1' and '0', not integers
- Edge case: empty grid $\rightarrow 0$

20. Final Cheat Sheet

Aspect	Details
When to use	Count connected components of '1's in grid
Main operations	Scan, DFS/BFS from each unvisited '1', sink
Complexity	$O(R \times C)$ time, $O(1)$ space (in-place)
Key code	<code>dfs(r,c){if(bad)return; grid[r][c]='0'; dfs(neighbors)}</code>
Edge cases	Empty, all '0', all '1', single row/col

7. BFS ON GRID

1. Overview

Breadth-First Search (BFS) on a grid is a level-order traversal algorithm that explores cells in increasing order of distance from a source. It is the foundation for shortest path problems in unweighted grids, multi-source problems, and many grid-based puzzles.

2. Intuition

Analogy: Imagine dropping a stone in a pond. The ripples expand outward in concentric circles. BFS works the same way: it explores all cells at distance 1 from the source, then all cells at distance 2, and so on.

Why it works: BFS uses a queue to ensure that cells are processed in order of their distance from the source. The first time a cell is visited, it is guaranteed to be via the shortest path (in an unweighted grid where each move has equal cost).

Step-by-step reasoning:

1. Start with the source cell in a queue.
2. Mark the source as visited with distance 0.
3. While the queue is not empty:
 - Pop the front cell (r, c).
 - For each unvisited neighbor:
 - Mark it as visited.
 - Set its distance = current distance + 1.
 - Push it into the queue.
4. The distance array now contains shortest distances from the source.

3. When to Use It

- When finding the shortest path in an unweighted grid
- When finding the minimum number of steps/moves to reach a target
- When exploring level by level (level-order traversal)
- When solving problems with "minimum moves" or "shortest distance"
- When using multi-source expansion (e.g., rotting oranges)

Trigger phrases: "shortest path", "minimum moves", "closest", "level order", "BFS", "distance to nearest", "rotten oranges"

4. When Not to Use It

- When the grid has weighted edges (use Dijkstra's algorithm)
- When you need to visit all cells but don't care about distance (use DFS for simplicity)
- When the grid is very large and memory is limited (BFS queue can be large)
- When you need to find paths with obstacles and complex constraints (use A* or Dijkstra)
- When the graph is a tree (DFS is simpler for tree traversal)

5. Core Concepts

Concept	Explanation	Why It Matters
Queue	Data structure storing cells to be processed	Ensures FIFO order, guaranteeing level-by-level traversal
Visited array	Tracks which cells have been visited	Prevents infinite loops and redundant processing
Distance array	Stores distance from source to each cell	BFS guarantees shortest distances in unweighted graphs
Level	Set of cells at the same distance from source	BFS processes one level at a time
Direction arrays	<code>dr = {-1,1,0,0}, dc = {0,0,-1,1}</code>	Standardized neighbor access
Multi-source BFS	Initialize queue with multiple sources	Solves problems like "distance to nearest 1"

6. Step-by-Step Algorithm

Standard BFS for shortest path:

1. Get `R`, `C` from grid dimensions.
2. Create `visited` (or `dist`) 2D array, initialized to `-1` (unvisited).
3. Create queue, push source `(sr, sc)`, set `dist[sr][sc] = 0`.
4. While queue not empty:
 - Pop `(r, c)`.
 - For each of 4 neighbors:
 - If neighbor is valid (in bounds, not visited, traversable):
 - Set `dist[nr][nc] = dist[r][c] + 1`.

- Push (nr, nc).
5. Return dist array (or distance to target).

7. Dry Run

Grid (0 = open, 1 = blocked):

```
0 0 0
0 1 0
0 0 0
```

Source: (0, 0), Target: (2, 2)

Queue	Current	Distance Grid	Visited
[(0,0)]	—	0 -1 -1 / -1 -1 -1 / -1 -1 -1	(0,0)
[(0,1), (1,0)]	(0,0)	0 1 -1 / 1 -1 -1 / -1 -1 -1	(0,0),(0,1),(1,0)
[(1,0), (0,2)]	(0,1)	0 1 2 / 1 -1 -1 / -1 -1 -1	+ (0,2)
[(0,2), (2,0)]	(1,0)	0 1 2 / 1 -1 -1 / 2 -1 -1	+ (2,0)
[(2,0), (1,2)]	(0,2)	0 1 2 / 1 -1 3 / 2 -1 -1	+ (1,2) (blocked, skip)
[(1,2)...]	(2,0)	(1,2) blocked	Skip
...	(1,2) blocked	Skip	—
Actually process (2,1) from (2,0):		0 1 2 / 1 -1 3 / 2 3 -1	+ (2,1)
From (2,1):		0 1 2 / 1 -1 3 / 2 3 4	+ (2,2)

Shortest distance from (0,0) to (2,2) = 4 (path: (0,0)→(0,1)→(0,2)→(2,0 is wrong... let me recompute)

Wait, let me redo this properly. The grid is:

```
0 0 0
0 1 0
0 0 0
```

From (0,0), the shortest path to (2,2) going around the blocker at (1,1): (0,0) → (0,1) → (0,2) → (1,2) → (2,2) = 4 steps

Or: (0,0) → (1,0) → (2,0) → (2,1) → (2,2) = 4 steps

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
```

```

// BFS for shortest path in unweighted grid
// Returns distance from (sr, sc) to all reachable cells
vector<vector<int>> bfsGrid(vector<vector<int>>& grid, int sr, int sc) {
    int R = grid.size(), C = grid[0].size();
    vector<vector<int>> dist(R, vector<int>(C, -1));
    queue<pair<int,int>> q;

    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};

    // Start from source
    q.push({sr, sc});
    dist[sr][sc] = 0;

    while (!q.empty()) {
        auto [r, c] = q.front(); q.pop();

        for (int k = 0; k < 4; k++) {
            int nr = r + dr[k], nc = c + dc[k];

            // Check bounds and if cell is traversable and unvisited
            if (nr >= 0 && nr < R && nc >= 0 && nc < C &&
                grid[nr][nc] == 0 && dist[nr][nc] == -1) {
                dist[nr][nc] = dist[r][c] + 1;
                q.push({nr, nc});
            }
        }
    }
    return dist;
}

// BFS to find shortest path to target
int shortestPath(vector<vector<int>>& grid, int sr, int sc, int tr, int tc) {
    auto dist = bfsGrid(grid, sr, sc);
    return dist[tr][tc]; // -1 if unreachable
}

// BFS with level-by-level processing (useful for "minimum steps")
int bfsLevels(vector<vector<int>>& grid, int sr, int sc, int tr, int tc) {
    int R = grid.size(), C = grid[0].size();
    vector<vector<bool>> vis(R, vector<bool>(C, false));
    queue<pair<int,int>> q;

    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};

    q.push({sr, sc});
    vis[sr][sc] = true;
    int steps = 0;

    while (!q.empty()) {
        int sz = q.size();
        while (sz-- > 0) {

```

```

        auto [r, c] = q.front(); q.pop();
        if (r == tr && c == tc) return steps;

        for (int k = 0; k < 4; k++) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr >= 0 && nr < R && nc >= 0 && nc < C &&
                grid[nr][nc] == 0 && !vis[nr][nc]) {
                vis[nr][nc] = true;
                q.push({nr, nc});
            }
        }
        steps++;
    }
    return -1; // unreachable
}

int main() {
    vector<vector<int>> grid = {
        {0, 0, 0},
        {0, 1, 0},
        {0, 0, 0}
    };
    cout << shortestPath(grid, 0, 0, 2, 2) << "\n"; // 4
    return 0;
}

```

9. Python Implementation

```

from typing import List
from collections import deque

def bfs_grid(grid: List[List[int]], sr: int, sc: int) -> List[List[int]]:
    R, C = len(grid), len(grid[0])
    dist = [[-1] * C for _ in range(R)]
    q = deque([(sr, sc)])
    dist[sr][sc] = 0
    dirs = [(-1,0), (1,0), (0,-1), (0,1)]

    while q:
        r, c = q.popleft()
        for dr, dc in dirs:
            nr, nc = r + dr, c + dc
            if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] == 0 and dist[nr][nc]
== -1:
                dist[nr][nc] = dist[r][c] + 1
                q.append((nr, nc))
    return dist

def shortest_path(grid: List[List[int]], sr: int, sc: int, tr: int, tc: int) ->
int:

```

```

    dist = bfs_grid(grid, sr, sc)
    return dist[tr][tc]

# Level-by-level BFS
def bfs_levels(grid: List[List[int]], sr: int, sc: int, tr: int, tc: int) -> int:
    R, C = len(grid), len(grid[0])
    vis = [[False] * C for _ in range(R)]
    q = deque([(sr, sc)])
    vis[sr][sc] = True
    dirs = [(-1,0), (1,0), (0,-1), (0,1)]
    steps = 0

    while q:
        for _ in range(len(q)):
            r, c = q.popleft()
            if r == tr and c == tc:
                return steps
            for dr, dc in dirs:
                nr, nc = r + dr, c + dc
                if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] == 0 and not
vis[nr][nc]:
                    vis[nr][nc] = True
                    q.append((nr, nc))
            steps += 1
    return -1

grid = [[0,0,0],[0,1,0],[0,0,0]]
print(shortest_path(grid, 0, 0, 2, 2)) # 4

```

10. Code Explanation

- **Distance array:** Initialized to -1 to indicate unvisited. When a cell is first visited, its distance is set to `dist[current] + 1`.
- **Queue:** Standard BFS queue. Each cell is processed once.
- **Direction arrays:** `dr` and `dc` encode the four cardinal directions. This is cleaner than four separate `if` statements.
- **Level-by-level BFS:** The inner loop `for (int sz = q.size(); sz--;` processes one entire level at a time. This is useful when you need to return the number of steps (levels) rather than the distance to each cell.
- **Traversable check:** `grid[nr][nc] == 0` ensures we only traverse through open cells. This condition can be customized based on the problem.

11. Complexity Analysis

Operation	Time	Space
BFS on R×C grid	O(R × C)	O(R × C) for dist/visited
Queue memory	O(R × C) worst-case	O(min(R, C)) typically
Processing each cell	O(R × C)	—

12. Common Patterns

Pattern	Identification	Approach	Example
Shortest path	"Minimum steps to reach"	Standard BFS with dist array	LeetCode 1091
Multi-source BFS	"Distance to nearest 1"	Initialize queue with all sources	LeetCode 542
Level-order BFS	"Minimum moves where each move is a step"	Level-by-level processing	LeetCode 994
BFS with state	"Minimum moves with extra state (keys, direction)"	BFS in state space	LeetCode 864
0-1 BFS	"Grid with 0-cost and 1-cost moves"	Deque instead of queue	Codeforces

13. Common Mistakes

- **Not marking visited when pushing to queue:** Mark visited at push time, not pop time. Otherwise, duplicates will be pushed.
- **Forgetting to check bounds:** Always validate neighbor coordinates.
- **Using DFS for shortest path:** DFS does not guarantee shortest path in unweighted graphs.
- **Not checking traversable cells:** Skip blocked cells (walls, obstacles).
- **Incorrect distance initialization:** Distance to source should be 0, not 1.
- **Queue overflow:** For very large grids, the queue can grow large.

14. Edge Cases

- Source equals target: Distance 0
- Target unreachable: Return -1
- Source is blocked: Should not happen (must be traversable)
- Grid with all cells blocked: Queue empties quickly
- Single row/column: Works fine
- Source at corner: Handled correctly

15. Variations

Variation	Description	Importance
Multi-source BFS	Start from multiple sources	High
0-1 BFS	Edges have weight 0 or 1, use deque	Medium
BFS with state (3D visited)	Include extra state (keys, health)	High (LeetCode 864)
A* search	Heuristic-guided BFS for faster pathfinding	Medium
Bidirectional BFS	BFS from both source and target	Medium

16. Related Algorithms/Data Structures

Related	Connection
DFS on grid	Different traversal order, no shortest path guarantee
Dijkstra's algorithm	BFS for weighted graphs
A* search	BFS with heuristic
Flood fill	BFS/DFS from a single source

17. Practice Problems

Easy

- **Shortest Path in Binary Matrix** (LeetCode 1091) — BFS in 8-directional grid
- **Flood Fill** (LeetCode 733) — BFS/DFS from source

Medium

- **01 Matrix** (LeetCode 542) — Multi-source BFS for distance to nearest 0
- **Rotting Oranges** (LeetCode 994) — Multi-source BFS with levels
- **Snakes and Ladders** (LeetCode 909) — BFS on 1D board mapped to 2D

Hard

- **Shortest Path in a Grid with Obstacles Elimination** (LeetCode 1293) — BFS with state (k remaining eliminations)
- **Minimum Cost to Make at Least One Valid Path** (LeetCode 1368) — 0-1 BFS

18. Interview Explanation

"BFS on a grid is used to find the shortest path in an unweighted grid. I use a queue and a distance array initialized to -1. Starting from the source, I process cells level by level. For each cell, I check its 4 neighbors and if they're traversable and unvisited, I set their distance to current distance + 1 and push them to the queue. BFS guarantees shortest paths because all edges have equal weight. The time complexity is $O(R \times C)$ and space is $O(R \times C)$ for the distance array."

19. Revision Notes

- Queue-based level-order traversal
- Guarantees shortest path in unweighted grid
- Mark visited when pushing to queue (not when popping)
- Distance array: init -1, source = 0
- 4-directional neighbors
- Level-by-level: process entire queue level for step counting
- Multi-source: push all sources initially
- $O(R \times C)$ time, $O(R \times C)$ space

20. Final Cheat Sheet

Aspect	Details
When to use	Shortest path in unweighted grid, level-order traversal
Main operations	Queue push/pop, visited check, distance update
Complexity	$O(R \times C)$ time, $O(R \times C)$ space
Key code	<pre>queue<pii> q; dist[src]=0; while(!q.empty()){auto[r,c]=q.front(); q.pop(); for neighbors...}</pre>
Edge cases	Source = target, unreachable, blocked source, single row/col

8. DFS ON GRID

1. Overview

Depth-First Search (DFS) on a grid is a recursive (or stack-based) traversal that explores as far as possible along each branch before backtracking. It is used for connectivity problems, path existence, and exhaustive search.

2. Intuition

Analogy: Imagine exploring a maze. You walk down one path until you hit a dead end, then backtrack to the last intersection and try the next path. This is exactly how DFS works — it goes deep first, then backtracks.

Why it works: DFS uses a stack (implicitly via recursion or explicitly) to remember the path. When it reaches a cell with no unvisited neighbors, it backtracks (pops from the stack) and tries the next option.

Step-by-step reasoning:

1. Start at the source cell.
2. Mark it as visited.
3. For each unvisited neighbor:
 - Recursively visit it.
4. When all neighbors are visited or invalid, return (backtrack).

3. When to Use It

- When checking if a path exists between two cells
- When counting connected components (islands)
- When exploring all possible paths (backtracking)
- When the problem requires exhaustive search
- When memory is limited (DFS uses less memory than BFS on average)
- When you need to process cells in a depth-first order

Trigger phrases: "path exists", "connected components", "explore all paths", "backtracking", "DFS", "island"

4. When Not to Use It

- When finding the shortest path (use BFS or Dijkstra)
- When the grid is very large and recursion depth is a concern (stack overflow)
- When you need level-by-level processing
- When the graph has cycles and you need to avoid infinite loops (use visited array)
- When you need the shortest path in an unweighted grid

5. Core Concepts

Concept	Explanation	Why It Matters
Recursion	Natural implementation of DFS	Uses call stack implicitly
Backtracking	Returning to previous state after exploring a branch	Essential for path-finding and search
Visited marking	Prevents revisiting cells	Prevents infinite loops
Stack (explicit)	Iterative DFS using explicit stack	Avoids recursion limit
Preorder/Postorder	Process before or after recursive calls	Determines traversal order
Connected components	Visiting all cells reachable from a source	Used for island counting

6. Step-by-Step Algorithm

Recursive DFS:

1. Define `dfs(r, c)`:
- If out of bounds or visited or blocked: return.
 - Mark `(r, c)` as visited.
 - Process `(r, c)` (if needed).
 - For each neighbor: call `dfs(nr, nc)`.
 - (Optional) Unmark `(r, c)` for backtracking problems.

Iterative DFS (using stack):

1. Push source `(r, c)` onto a stack.
2. While stack is not empty:
- Pop `(r, c)`.
 - If visited or blocked: continue.
 - Mark as visited.
 - Process `(r, c)`.
 - Push all unvisited neighbors onto the stack.

7. Dry Run

Grid:

```
0 0 0
0 1 0
```

0 0 0

DFS from (0,0) visiting order (right→down→left→up priority):

Call Stack	Current	Action	Visited
dfs(0,0)	(0,0)	Mark visited	(0,0)
dfs(0,0)→dfs(0,1)	(0,1)	Mark visited	(0,0),(0,1)
dfs(0,0)→dfs(0,1)→dfs(0,2)	(0,2)	Mark visited	(0,0),(0,1),(0,2)
dfs(0,0)→dfs(0,1)→dfs(0,2)→dfs(1,2)	(1,2)	Mark visited	+ (1,2)
dfs(0,0)→dfs(0,1)→dfs(0,2)→dfs(1,2)→dfs(2,2)	(2,2)	Mark visited	+ (2,2)
dfs(0,0)→dfs(0,1)→dfs(0,2)→dfs(1,2)→dfs(2,2)→dfs(2,1)	(2,1)	Mark visited	+ (2,1)
Backtracking...	(2,0)	Mark visited via (2,1)	+ (2,0)
Backtrack to (1,0) via (2,0)	(1,0)	Mark visited	+ (1,0)

DFS visiting order: (0,0) → (0,1) → (0,2) → (1,2) → (2,2) → (2,1) → (2,0) → (1,0)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Recursive DFS for connectivity
void dfs(vector<vector<int>>& grid, vector<vector<bool>>& vis, int r, int c) {
    int R = grid.size(), C = grid[0].size();
    if (r < 0 || r >= R || c < 0 || c >= C) return;
    if (vis[r][c] || grid[r][c] == 1) return; // visited or blocked

    vis[r][c] = true;
    cout << "Visiting: (" << r << ", " << c << ")\n";

    // Recurse to neighbors
    dfs(grid, vis, r - 1, c); // up
    dfs(grid, vis, r + 1, c); // down
    dfs(grid, vis, r, c - 1); // left
    dfs(grid, vis, r, c + 1); // right
}

// Iterative DFS using stack
void dfsIterative(vector<vector<int>>& grid, int sr, int sc) {
    int R = grid.size(), C = grid[0].size();
    vector<vector<bool>> vis(R, vector<bool>(C, false));
    stack<pair<int,int>> st;

    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};
```

```

    st.push({sr, sc});

    while (!st.empty()) {
        auto [r, c] = st.top(); st.pop();

        if (r < 0 || r >= R || c < 0 || c >= C) continue;
        if (vis[r][c] || grid[r][c] == 1) continue;

        vis[r][c] = true;
        cout << "Visiting: (" << r << ", " << c << ")\n";

        // Push neighbors in reverse order to simulate same order as recursive
        for (int k = 3; k >= 0; k--)
            st.push({r + dr[k], c + dc[k]});
    }
}

// DFS path existence check
bool hasPath(vector<vector<int>>& grid, int sr, int sc, int tr, int tc) {
    int R = grid.size(), C = grid[0].size();
    vector<vector<bool>> vis(R, vector<bool>(C, false));

    function<bool(int,int)> dfs = [&](int r, int c) -> bool {
        if (r < 0 || r >= R || c < 0 || c >= C) return false;
        if (vis[r][c] || grid[r][c] == 1) return false;
        if (r == tr && c == tc) return true;

        vis[r][c] = true;
        return dfs(r-1, c) || dfs(r+1, c) || dfs(r, c-1) || dfs(r, c+1);
    };

    return dfs(sr, sc);
}

int main() {
    vector<vector<int>> grid = {{0,0,0},{0,1,0},{0,0,0}};
    vector<vector<bool>> vis(3, vector<bool>(3, false));
    dfs(grid, vis, 0, 0);
    // DFS order: (0,0), (0,1), (0,2), (1,2), (2,2), (2,1), (2,0), (1,0)
    cout << hasPath(grid, 0, 0, 2, 2) << "\n"; // 1 (true)
    return 0;
}

```

9. Python Implementation

```

from typing import List

def dfs_grid(grid: List[List[int]], vis: List[List[bool]], r: int, c: int):
    R, C = len(grid), len(grid[0])
    if r < 0 or r >= R or c < 0 or c >= C:

```

```

        return
    if vis[r][c] or grid[r][c] == 1:
        return

    vis[r][c] = True
    print(f"Visiting: ({r},{c})")

    dfs_grid(grid, vis, r - 1, c)
    dfs_grid(grid, vis, r + 1, c)
    dfs_grid(grid, vis, r, c - 1)
    dfs_grid(grid, vis, r, c + 1)

def has_path(grid: List[List[int]], sr: int, sc: int, tr: int, tc: int) -> bool:
    R, C = len(grid), len(grid[0])
    vis = [[False] * C for _ in range(R)]

    def dfs(r, c):
        if r < 0 or r >= R or c < 0 or c >= C:
            return False
        if vis[r][c] or grid[r][c] == 1:
            return False
        if r == tr and c == tc:
            return True
        vis[r][c] = True
        return (dfs(r-1, c) or dfs(r+1, c) or
                dfs(r, c-1) or dfs(r, c+1))

    return dfs(sr, sc)

# Iterative DFS
def dfs_iterative(grid: List[List[int]], sr: int, sc: int):
    R, C = len(grid), len(grid[0])
    vis = [[False] * C for _ in range(R)]
    stack = [(sr, sc)]
    dirs = [(-1,0), (1,0), (0,-1), (0,1)]

    while stack:
        r, c = stack.pop()
        if r < 0 or r >= R or c < 0 or c >= C:
            continue
        if vis[r][c] or grid[r][c] == 1:
            continue
        vis[r][c] = True
        print(f"Visiting: ({r},{c})")
        for dr, dc in reversed(dirs):
            stack.append((r + dr, c + dc))

grid = [[0,0,0],[0,1,0],[0,0,0]]
vis = [[False]*3 for _ in range(3)]
dfs_grid(grid, vis, 0, 0)
print(has_path(grid, 0, 0, 2, 2)) # True

```

10. Code Explanation

- **Recursive DFS:** The simplest form. Base cases check bounds, visited, and blocked cells. The function marks the current cell, processes it, then recurses to all 4 neighbors.
- **Iterative DFS:** Uses an explicit stack. The order of visiting depends on the order neighbors are pushed. Pushing in reverse order of the recursive version simulates the same traversal.
- **Path existence:** Uses DFS with early exit. The `||` operator short-circuits: if a path is found through one neighbor, the remaining neighbors are not explored.
- **Visited array:** Using a separate `visited` array preserves the original grid. This is preferred over the "sink" technique when the grid should not be modified.

11. Complexity Analysis

Approach	Time	Space
Recursive DFS	$O(R \times C)$	$O(R \times C)$ worst-case recursion stack
Iterative DFS (stack)	$O(R \times C)$	$O(R \times C)$ worst-case stack size
Path existence	$O(R \times C)$	$O(R \times C)$

12. Common Patterns

Pattern	Identification	Approach	Example
Connected components	"Count islands"	DFS from each unvisited cell	LeetCode 200
Path existence	"Is there a path?"	DFS with early exit	LeetCode 695
Backtracking	"Find all paths"	DFS with unmarking	LeetCode 79
Coloring	"Color connected regions"	DFS with color assignment	LeetCode 733
Cycle detection	"Detect cycle in grid"	DFS with parent tracking	LeetCode

13. Common Mistakes

- **Stack overflow:** Recursive DFS can overflow for large grids (e.g., 1000×1000). Use iterative DFS or BFS.
- **Not marking visited before recursion:** Mark visited at the start of the function, not after the recursive calls.
- **Forgetting to unmark for backtracking:** In path-finding problems that require all paths, you must unmark after recursion.
- **Incorrect neighbor order:** The order of neighbor visits changes the traversal but not the correctness.
- **Using DFS for shortest path:** DFS does not guarantee the shortest path.

14. Edge Cases

- Empty grid: Return immediately
- 1×1 grid: Single cell, visited immediately
- Source is blocked: Return immediately
- Grid with all cells blocked: No traversal

- Target unreachable: Returns false
- Grid with cycles: Visited array prevents infinite loops

15. Variations

Variation	Description	Importance
Backtracking on grid	Find all paths, unmark after recursion	High (LeetCode 79)
DFS with state	Include direction, remaining steps in state	Medium
Iterative deepening DFS	DFS with depth limit, combines BFS and DFS	Low
DFS for topological order	Not applicable on grid (grid is undirected)	N/A

16. Related Algorithms/Data Structures

Related	Connection
BFS on grid	Different traversal order, BFS for shortest path
Flood fill	Same as DFS/BFS from source
Backtracking	DFS with unmarking
Union-Find	Alternative for connectivity queries

17. Practice Problems

Easy

- **Flood Fill** (LeetCode 733) — DFS from source
- **Number of Islands** (LeetCode 200) — Count components with DFS

Medium

- **Word Search** (LeetCode 79) — Backtracking DFS on grid
- **Max Area of Island** (LeetCode 695) — DFS with size tracking
- **Surrounded Regions** (LeetCode 130) — DFS from boundary

Hard

- **Word Search II** (LeetCode 212) — DFS + Trie
- **Longest Increasing Path in a Matrix** (LeetCode 329) — DFS with memoization

18. Interview Explanation

"DFS on a grid is a recursive traversal that explores as deep as possible along each path before backtracking. I implement it with a recursive function that checks bounds, visited status, and blocked cells, then recurses to the 4 neighbors. The key advantage of DFS is its simplicity and low memory usage — it only stores the current path on the call stack. However, for very large grids, recursion depth

can be a problem, so I use an explicit stack or switch to BFS. DFS is ideal for connectivity problems, path existence, and backtracking, but not for shortest paths."

19. Revision Notes

- Recursive: check bounds + visited + blocked → mark → recurse
- Iterative: use explicit stack
- Stack overflow risk for large grids
- Not suitable for shortest path
- Mark visited before recursion (not after)
- For backtracking: unmark after recursion
- $O(R \times C)$ time, $O(R \times C)$ space (worst-case stack)

20. Final Cheat Sheet

Aspect	Details
When to use	Connectivity, path existence, backtracking, exhaustive search
Main operations	Recursive call, visited check, neighbor traversal
Complexity	$O(R \times C)$ time, $O(R \times C)$ space (stack)
Key code	<code>dfs(r,c){if(bad)return; vis[r][c]=true; dfs(neighbors);}</code>
Edge cases	Empty, blocked source, large grid (stack overflow)

9. SHORTEST PATH IN GRID

1. Overview

Finding the shortest path in a grid involves determining the minimum number of steps (or minimum cost) to travel from a source cell to a target cell while navigating obstacles. Depending on the grid properties (unweighted, weighted, with obstacles), different algorithms are used: BFS, Dijkstra, 0-1 BFS, or A*.

2. Intuition

Analogy: You're in a city with blocks (grid cells). Some blocks are parks (open), some are buildings (blocked). Each step along a street takes 1 minute. You want the fastest route to a destination. This is the shortest path problem on a grid.

Why different algorithms?

- **BFS:** When every move costs 1 (unweighted), BFS from source guarantees shortest path.
- **Dijkstra:** When moves have different costs (e.g., 1 on road, 5 on mud), we need a priority queue.
- **0-1 BFS:** When costs are only 0 or 1, we can use a deque for $O(R \times C)$ time.
- **A*:** When we need to find the path faster using a heuristic (Manhattan distance).

Step-by-step reasoning (BFS for unweighted grid):

1. Start from source with distance 0.
2. Use a queue. Process cells in order of distance.
3. For each cell, check all 4 neighbors.
4. If a neighbor is traversable and not yet visited, set its distance = current + 1.
5. Stop when we reach the target.

3. When to Use It

- When the problem asks for "shortest path", "minimum steps", "minimum moves"
- When navigating a grid with obstacles
- When the grid has weighted cells (different costs for different terrain)
- When you need to find the optimal path with constraints (k obstacle removals, etc.)

Trigger phrases: "shortest path", "minimum steps", "minimum number of moves", "shortest distance", "optimal path", "minimum cost"

4. When Not to Use It

- When you only need to know if a path exists (use DFS for simplicity)
- When the grid is very small (brute force is fine)
- When you need all paths between two points (use DFS)
- When the graph is a tree (tree DP is better)
- When the grid is huge and you need real-time performance (use A* with heuristic)

5. Core Concepts

Concept	Explanation	Why It Matters
Unweighted vs weighted	All moves cost 1 vs different costs per cell	Determines algorithm choice (BFS vs Dijkstra)
Obstacles	Cells that cannot be traversed	Must be skipped during traversal
Distance array	Stores shortest distance from source to each cell	BFS fills this in order of distance
Visited array	Prevents revisiting cells	Essential for correctness
Path reconstruction	Storing parent pointers to reconstruct the path	Required when the path itself is needed
Manhattan distance	`	$r1-r2$

6. Step-by-Step Algorithm

BFS for shortest path (unweighted):

1. Initialize $dist[R][C] = INF$, $dist[sr][sc] = 0$.
2. Initialize queue, push (sr, sc) .
3. While queue not empty:
 - Pop (r, c) .

- If $(r, c) == (tr, tc)$: return $dist[r][c]$.
- For each neighbor (nr, nc) :
 - If in bounds, traversable, and $dist[nr][nc] > dist[r][c] + 1$:
 - $dist[nr][nc] = dist[r][c] + 1$
 - Push (nr, nc) .
- 4. Return -1 (unreachable).

Dijkstra for weighted grid:

1. Same as BFS but use priority queue (min-heap) instead of queue.
2. Process cells in order of smallest distance.
3. Relaxation: if $dist[nr][nc] > dist[r][c] + cost[nr][nc]$, update and push.

7. Dry Run

Grid (0 = open, 1 = blocked):

```
0 0 0
0 1 0
0 0 0
```

Source: $(0, 0)$, Target: $(2, 2)$

BFS distance array evolution:

Step	Queue	Distance Grid
Init	[(0,0)]	0 ∞ ∞ / ∞ ∞ ∞ / ∞ ∞ ∞
1	Pop (0,0), push (0,1),(1,0)	0 1 ∞ / 1 ∞ ∞ / ∞ ∞ ∞
2	Pop (0,1), push (0,2)	0 1 2 / 1 ∞ ∞ / ∞ ∞ ∞
3	Pop (1,0), push (2,0)	0 1 2 / 1 ∞ ∞ / 2 ∞ ∞
4	Pop (0,2), push (1,2)	0 1 2 / 1 ∞ 3 / 2 ∞ ∞
5	Pop (2,0), push (2,1)	0 1 2 / 1 ∞ 3 / 2 3 ∞
6	Pop (2,1), push (2,2)	0 1 2 / 1 ∞ 3 / 2 3 4

Shortest distance: 4

Path: $(0,0) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (1,2) \rightarrow (2,2)$ or $(0,0) \rightarrow (1,0) \rightarrow (2,0) \rightarrow (2,1) \rightarrow (2,2)$

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// BFS for shortest path in unweighted grid
```

```

int shortestPathBFS(vector<vector<int>>& grid, int sr, int sc, int tr, int tc) {
    int R = grid.size(), C = grid[0].size();
    vector<vector<int>> dist(R, vector<int>(C, INT_MAX));
    queue<pair<int,int>> q;

    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};

    dist[sr][sc] = 0;
    q.push({sr, sc});

    while (!q.empty()) {
        auto [r, c] = q.front(); q.pop();

        if (r == tr && c == tc) return dist[r][c];

        for (int k = 0; k < 4; k++) {
            int nr = r + dr[k], nc = c + dc[k];

            if (nr >= 0 && nr < R && nc >= 0 && nc < C &&
                grid[nr][nc] == 0 && dist[nr][nc] > dist[r][c] + 1) {
                dist[nr][nc] = dist[r][c] + 1;
                q.push({nr, nc});
            }
        }
    }
    return -1;
}

```

// Dijkstra for weighted grid

```

int shortestPathDijkstra(vector<vector<int>>& grid, int sr, int sc, int tr, int
tc) {
    int R = grid.size(), C = grid[0].size();
    vector<vector<int>> dist(R, vector<int>(C, INT_MAX));
    // Min-heap: (distance, row, col)
    priority_queue<tuple<int,int,int>, vector<tuple<int,int,int>>, greater<>> pq;

    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};

    dist[sr][sc] = grid[sr][sc]; // cost of starting cell
    pq.push({dist[sr][sc], sr, sc});

    while (!pq.empty()) {
        auto [d, r, c] = pq.top(); pq.pop();

        if (d > dist[r][c]) continue; // stale entry
        if (r == tr && c == tc) return d;

        for (int k = 0; k < 4; k++) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr >= 0 && nr < R && nc >= 0 && nc < C) {
                int nd = d + grid[nr][nc]; // cost of moving into this cell
                if (nd < dist[nr][nc]) {

```

```

        dist[nr][nc] = nd;
        pq.push({nd, nr, nc});
    }
}
}
}
return -1;
}

// 0-1 BFS (when costs are 0 or 1)
int shortestPath01BFS(vector<vector<int>>& grid, int sr, int sc, int tr, int tc) {
    int R = grid.size(), C = grid[0].size();
    vector<vector<int>> dist(R, vector<int>(C, INT_MAX));
    deque<pair<int,int>> dq;

    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};

    dist[sr][sc] = 0;
    dq.push_front({sr, sc});

    while (!dq.empty()) {
        auto [r, c] = dq.front(); dq.pop_front();

        if (r == tr && c == tc) return dist[r][c];

        for (int k = 0; k < 4; k++) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr >= 0 && nr < R && nc >= 0 && nc < C) {
                int cost = grid[nr][nc]; // 0 or 1
                if (dist[nr][nc] > dist[r][c] + cost) {
                    dist[nr][nc] = dist[r][c] + cost;
                    if (cost == 0) dq.push_front({nr, nc});
                    else dq.push_back({nr, nc});
                }
            }
        }
    }
    return -1;
}

// Path reconstruction
vector<pair<int,int>> reconstructPath(vector<vector<int>>& parent, int tr, int tc)
{
    vector<pair<int,int>> path;
    int r = tr, c = tc;
    while (r != -1) {
        path.push_back({r, c});
        int p = parent[r][c];
        // parent encoded as: r * C + c (needs separate impl)
        // Simplified: assume parent stores r*C + c
        int pr = p / 1000, pc = p % 1000; // HACK: use proper encoding
        r = pr; c = pc;
    }
}

```

```

        reverse(path.begin(), path.end());
        return path;
    }

    int main() {
        vector<vector<int>> grid = {{0,0,0},{0,1,0},{0,0,0}};
        cout << shortestPathBFS(grid, 0, 0, 2, 2) << "\n"; // 4

        // Weighted grid (cost = cell value)
        vector<vector<int>> wgrid = {{1,2,1},{1,5,1},{1,1,1}};
        cout << shortestPathDijkstra(wgrid, 0, 0, 2, 2) << "\n";
        return 0;
    }

```

9. Python Implementation

```

from typing import List, Tuple
from collections import deque
import heapq

def shortest_path_bfs(grid: List[List[int]], sr: int, sc: int, tr: int, tc: int) -
> int:
    R, C = len(grid), len(grid[0])
    dist = [[float('inf')] * C for _ in range(R)]
    q = deque([(sr, sc)])
    dist[sr][sc] = 0
    dirs = [(-1,0), (1,0), (0,-1), (0,1)]

    while q:
        r, c = q.popleft()
        if r == tr and c == tc:
            return dist[r][c]
        for dr, dc in dirs:
            nr, nc = r + dr, c + dc
            if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] == 0:
                if dist[nr][nc] > dist[r][c] + 1:
                    dist[nr][nc] = dist[r][c] + 1
                    q.append((nr, nc))

    return -1

def shortest_path_dijkstra(grid: List[List[int]], sr: int, sc: int, tr: int, tc:
int) -> int:
    R, C = len(grid), len(grid[0])
    dist = [[float('inf')] * C for _ in range(R)]
    pq = [(grid[sr][sc], sr, sc)]
    dist[sr][sc] = grid[sr][sc]
    dirs = [(-1,0), (1,0), (0,-1), (0,1)]

    while pq:
        d, r, c = heapq.heappop(pq)
        if d > dist[r][c]:

```

```

        continue
    if r == tr and c == tc:
        return d
    for dr, dc in dirs:
        nr, nc = r + dr, c + dc
        if 0 <= nr < R and 0 <= nc < C:
            nd = d + grid[nr][nc]
            if nd < dist[nr][nc]:
                dist[nr][nc] = nd
                heapq.heappush(pq, (nd, nr, nc))
    return -1

def shortest_path_01bfs(grid: List[List[int]], sr: int, sc: int, tr: int, tc: int)
-> int:
    R, C = len(grid), len(grid[0])
    dist = [[float('inf')] * C for _ in range(R)]
    dq = deque([(sr, sc)])
    dist[sr][sc] = 0
    dirs = [(-1,0), (1,0), (0,-1), (0,1)]

    while dq:
        r, c = dq.popleft()
        if r == tr and c == tc:
            return dist[r][c]
        for dr, dc in dirs:
            nr, nc = r + dr, c + dc
            if 0 <= nr < R and 0 <= nc < C:
                cost = grid[nr][nc]
                if dist[nr][nc] > dist[r][c] + cost:
                    dist[nr][nc] = dist[r][c] + cost
                    if cost == 0:
                        dq.appendleft((nr, nc))
                    else:
                        dq.append((nr, nc))
    return -1

grid = [[0,0,0],[0,1,0],[0,0,0]]
print(shortest_path_bfs(grid, 0, 0, 2, 2)) # 4

```

10. Code Explanation

- **BFS version:** Uses a queue and distance array. The `dist[nr][nc] > dist[r][c] + 1` check ensures we only update if we found a shorter path. Since BFS processes cells in order of distance, this check is technically redundant for unweighted BFS but is good practice.
- **Dijkstra version:** Uses a priority queue (min-heap). The `if (d > dist[r][c]) continue;` line skips stale entries (cells that were already updated with a shorter distance). This is the standard Dijkstra optimization.
- **0-1 BFS:** Uses a deque. When the cost is 0, we push to the front (process it immediately). When the cost is 1, we push to the back. This maintains the invariant that deque is always sorted by distance.
- **Path reconstruction:** Parent pointers are stored during traversal. At the end, we backtrack from target to source.

11. Complexity Analysis

Algorithm	Time	Space
BFS (unweighted)	$O(R \times C)$	$O(R \times C)$
Dijkstra (weighted)	$O(R \times C \times \log(R \times C))$	$O(R \times C)$
0-1 BFS	$O(R \times C)$	$O(R \times C)$
A*	$O(R \times C)$ (with good heuristic)	$O(R \times C)$

12. Common Patterns

Pattern	Identification	Approach	Example
Shortest path with obstacles	"Grid with walls"	BFS, skip blocked cells	LeetCode 1091
Shortest path with weighted cells	"Cell has cost to enter"	Dijkstra	LeetCode 1368
Shortest path with k removals	"Remove k obstacles"	BFS with state (k remaining)	LeetCode 1293
Shortest path with keys	"Collect keys to open doors"	BFS with bitmask state	LeetCode 864
Minimum time to reach	"Time-dependent costs"	Dijkstra with time dimension	Codeforces

13. Common Mistakes

- **Using DFS for shortest path:** DFS does not guarantee shortest path in an unweighted graph.
- **Not marking visited when pushing:** Mark visited at push time for BFS, not pop time.
- **Using BFS for weighted grid:** BFS assumes all edges have equal weight.
- **Forgetting the heuristic for A*:** Without a proper heuristic, A* degrades to Dijkstra.
- **Integer overflow:** Use `INT_MAX` or `float('inf')` for initialization.
- **Not handling unreachable target:** Return -1 or INF.

14. Edge Cases

- Source equals target: Distance 0
- Target unreachable: Return -1
- Source is blocked: Should not be (check problem constraints)
- Grid with no obstacles: Straight path
- Grid with all cells blocked (except source): Unreachable
- Single row or column: Handled correctly
- Large grid with many obstacles: BFS may be slow

15. Variations

Variation	Description	Importance
Shortest path with obstacle removal	Can remove up to k obstacles	High (LeetCode 1293)
Shortest path with keys and doors	Need keys to open doors	High (LeetCode 864)
Minimum path sum	Find path with minimum sum of cell values	High (LeetCode 64)
Shortest path in a grid with portals	Teleportation between cells	Medium
A* search	Heuristic-guided for faster search	Medium

16. Related Algorithms/Data Structures

Related	Connection
BFS	Shortest path in unweighted grid
Dijkstra	Shortest path in weighted grid
0-1 BFS	Special case of Dijkstra for 0/1 weights
A* search	BFS/Dijkstra with heuristic
Dynamic Programming	For DAG-like grid problems (minimum path sum)

17. Practice Problems

Easy

- **Shortest Path in Binary Matrix** (LeetCode 1091) — 8-directional BFS
- **Minimum Path Sum** (LeetCode 64) — DP for minimum sum path

Medium

- **Shortest Path in a Grid with Obstacles Elimination** (LeetCode 1293) — BFS with state
- **Minimum Cost to Make at Least One Valid Path** (LeetCode 1368) — 0-1 BFS
- **The Maze** (LeetCode 490) — BFS with rolling ball mechanics

Hard

- **Shortest Path to Get All Keys** (LeetCode 864) — BFS with bitmask
- **Minimum Path Cost in a Grid** (LeetCode 2304) — DP with path cost

18. Interview Explanation

"For finding the shortest path in a grid, I first check if the grid is unweighted or weighted. For unweighted grids, I use BFS which guarantees the shortest path in $O(R \times C)$ time. I maintain a distance array and a queue, processing cells level by level. For weighted grids, I use Dijkstra's algorithm with a priority queue, which handles different costs for different cells. For the special case where costs are only

0 or 1, I use 0-1 BFS with a deque for $O(R \times C)$ performance. Common extensions include obstacle removal, keys and doors, and path reconstruction using parent pointers."

19. Revision Notes

- Unweighted: BFS with queue, $O(R \times C)$
- Weighted: Dijkstra with priority queue, $O(R \times C \log(R \times C))$
- 0-1 weights: 0-1 BFS with deque, $O(R \times C)$
- Distance array: init INF, update on shorter path
- Path reconstruction: store parent pointers
- BFS mark visited when pushing (not popping)
- Edge case: unreachable $\rightarrow$ return -1

20. Final Cheat Sheet

Aspect	Details
When to use	"Minimum steps", "shortest path" in grid
Main operations	BFS (unweighted), Dijkstra (weighted), 0-1 BFS (0/1 costs)
Complexity	$O(R \times C)$ BFS, $O(R \times C \log(R \times C))$ Dijkstra
Key code	BFS: <code>queue + dist array + 4 neighbors</code>
Edge cases	Source = target, unreachable, blocked source, single row/col

10. MULTI-SOURCE BFS

1. Overview

Multi-source BFS is a variant of BFS where the initial queue is seeded with multiple source cells instead of a single source. It computes the shortest distance from any of the source cells to every other cell. It is also called "wavefront propagation" or "grassfire transform."

2. Intuition

Analogy: Imagine multiple fires starting simultaneously in a forest. The fires spread outward at the same speed. The time it takes for a point to catch fire is the distance to the nearest fire source. Multi-source BFS simulates this exact process.

Why it works: By initializing the queue with all source cells at distance 0, BFS proceeds level by level. Since all sources start at the same distance, the first time a cell is visited, it is reached via the shortest path from the nearest source. This is equivalent to adding a virtual super-source connected to all actual sources with 0-cost edges.

Step-by-step reasoning:

1. Identify all source cells (e.g., all '0' cells in a 01-matrix).
2. Push all source cells into the queue with distance 0.

3. Run standard BFS.
4. The distance array now contains the shortest distance from any source to each cell.

3. When to Use It

- When computing the distance from each cell to the nearest source (e.g., nearest 0, nearest 1)
- When simulating simultaneous spread (e.g., rotting oranges, fire spreading)
- When the problem asks for "distance to nearest X" or "closest Y"
- When multiple starting points expand simultaneously
- When computing the Voronoi diagram on a grid (partitioning by nearest source)

Trigger phrases: "distance to nearest", "closest", "multi-source", "simultaneously", "spread from all", "nearest 0", "rotting oranges"

4. When Not to Use It

- When there is only one source (use standard BFS)
- When you need the distance from a specific source (use standard BFS from that source)
- When the grid is very large and memory is a concern (BFS queue can be huge)
- When you need to compute distances for multiple different source sets (compute once and reuse)

5. Core Concepts

Concept	Explanation	Why It Matters
Multiple sources	All initial cells pushed to queue with distance 0	The core idea of multi-source BFS
Virtual super-source	Conceptual single source connected to all actual sources	Helps reason about correctness
Distance to nearest	Each cell's distance = min distance to any source	The output of the algorithm
Wavefront	The expanding frontier of visited cells	All cells at same distance form a wavefront
Simultaneous expansion	All sources expand at the same rate	Ensures first visit = shortest path from nearest source

6. Step-by-Step Algorithm

1. Initialize `dist[R][C] = INT_MAX`.
2. Initialize queue.
3. For each cell `(r, c)`:
 - If it is a source (e.g., `grid[r][c] == 0`):
 - Set `dist[r][c] = 0`.
 - Push `(r, c)` into queue.
4. While queue not empty:
 - Pop `(r, c)`.

- For each neighbor (nr, nc) :
 - If in bounds and $dist[nr][nc] > dist[r][c] + 1$:
 - Set $dist[nr][nc] = dist[r][c] + 1$.
 - Push (nr, nc) .

5. Return $dist$ array.

7. Dry Run

Grid:

```
0 0 1
0 1 1
1 1 1
```

Sources: all cells with value 0 at (0,0), (0,1), (1,0)

Step	Queue	Distance Grid
Init	[(0,0),(0,1),(1,0)]	0 0 ∞ / 0 ∞ ∞ / ∞ ∞ ∞
1	Pop (0,0), push (0,1 skip), (1,0 skip)	0 0 ∞ / 0 ∞ ∞ / ∞ ∞ ∞
2	Pop (0,1), push (0,2)	0 0 1 / 0 ∞ ∞ / ∞ ∞ ∞
3	Pop (1,0), push (2,0)	0 0 1 / 0 ∞ ∞ / 1 ∞ ∞
4	Pop (0,2), push (1,2)	0 0 1 / 0 2 2 / 1 ∞ ∞
5	Pop (2,0), push (2,1)	0 0 1 / 0 2 2 / 1 2 ∞
6	Pop (1,2), push (2,2)	0 0 1 / 0 2 2 / 1 2 3
7	Pop (2,1), push (2,2 skip)	0 0 1 / 0 2 2 / 1 2 3
8	Pop (2,2)	Done

Result: Each cell shows distance to nearest 0.

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Multi-source BFS: distance from each cell to nearest 0
vector<vector<int>> updateMatrix(vector<vector<int>>& grid) {
    int R = grid.size(), C = grid[0].size();
    vector<vector<int>> dist(R, vector<int>(C, INT_MAX));
    queue<pair<int,int>> q;

    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};
```

```

// Initialize: push all zero cells as sources
for (int i = 0; i < R; i++) {
    for (int j = 0; j < C; j++) {
        if (grid[i][j] == 0) {
            dist[i][j] = 0;
            q.push({i, j});
        }
    }
}

// BFS
while (!q.empty()) {
    auto [r, c] = q.front(); q.pop();

    for (int k = 0; k < 4; k++) {
        int nr = r + dr[k], nc = c + dc[k];
        if (nr >= 0 && nr < R && nc >= 0 && nc < C) {
            if (dist[nr][nc] > dist[r][c] + 1) {
                dist[nr][nc] = dist[r][c] + 1;
                q.push({nr, nc});
            }
        }
    }
}
return dist;
}

// Rotting Oranges: multi-source BFS with level tracking
int orangesRotting(vector<vector<int>>& grid) {
    int R = grid.size(), C = grid[0].size();
    queue<pair<int,int>> q;
    int fresh = 0;

    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};

    // Initialize: push all rotten oranges, count fresh
    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            if (grid[i][j] == 2) q.push({i, j});
            else if (grid[i][j] == 1) fresh++;
        }
    }

    if (fresh == 0) return 0; // no fresh oranges

    int minutes = 0;

    // Level-by-level BFS
    while (!q.empty() && fresh > 0) {
        int sz = q.size();
        minutes++;
        while (sz-- > 0) {

```

```

        auto [r, c] = q.front(); q.pop();
        for (int k = 0; k < 4; k++) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr][nc] == 1) {
                grid[nr][nc] = 2; // rot this orange
                fresh--;
                q.push({nr, nc});
            }
        }
    }
}

return fresh == 0 ? minutes : -1;
}

int main() {
    vector<vector<int>> grid = {{0,0,1},{0,1,1},{1,1,1}};
    auto dist = updateMatrix(grid);
    for (auto& row : dist) {
        for (int x : row) cout << x << " ";
        cout << "\n";
    }
    // Output:
    // 0 0 1
    // 0 1 2
    // 1 2 3

    vector<vector<int>> oranges = {{2,1,1},{1,1,0},{0,1,1}};
    cout << orangesRotting(oranges) << "\n"; // 4
    return 0;
}

```

9. Python Implementation

```

from typing import List
from collections import deque

def updateMatrix(grid: List[List[int]]) -> List[List[int]]:
    R, C = len(grid), len(grid[0])
    dist = [[float('inf')] * C for _ in range(R)]
    q = deque()
    dirs = [(-1,0), (1,0), (0,-1), (0,1)]

    # Initialize: push all zero cells
    for i in range(R):
        for j in range(C):
            if grid[i][j] == 0:
                dist[i][j] = 0
                q.append((i, j))

    while q:

```

```

        r, c = q.popleft()
        for dr, dc in dirs:
            nr, nc = r + dr, c + dc
            if 0 <= nr < R and 0 <= nc < C:
                if dist[nr][nc] > dist[r][c] + 1:
                    dist[nr][nc] = dist[r][c] + 1
                    q.append((nr, nc))

    return dist

def orangesRotting(grid: List[List[int]]) -> int:
    R, C = len(grid), len(grid[0])
    q = deque()
    fresh = 0
    dirs = [(-1,0), (1,0), (0,-1), (0,1)]

    for i in range(R):
        for j in range(C):
            if grid[i][j] == 2:
                q.append((i, j))
            elif grid[i][j] == 1:
                fresh += 1

    if fresh == 0:
        return 0

    minutes = 0
    while q and fresh > 0:
        for _ in range(len(q)):
            r, c = q.popleft()
            for dr, dc in dirs:
                nr, nc = r + dr, c + dc
                if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] == 1:
                    grid[nr][nc] = 2
                    fresh -= 1
                    q.append((nr, nc))
        minutes += 1

    return minutes if fresh == 0 else -1

grid = [[0,0,1],[0,1,1],[1,1,1]]
print(updateMatrix(grid))

```

10. Code Explanation

- **Initialization loop:** All source cells (0's) are identified and pushed to the queue with distance 0. This is the only difference from standard BFS.
- **BFS loop:** Standard BFS with distance update. The `dist[nr][nc] > dist[r][c] + 1` check ensures we only update if we find a shorter path.
- **Rotting Oranges variant:** Uses level-by-level BFS to count minutes. Each level corresponds to one minute. The `fresh` counter tracks remaining fresh oranges. If fresh oranges remain after BFS, return -1.

- **Correctness:** The first time a cell is visited, it's via the shortest path from the nearest source. This is guaranteed because all sources start at distance 0 and BFS processes cells in order of distance.

11. Complexity Analysis

Operation	Time	Space
Multi-source BFS	$O(R \times C)$	$O(R \times C)$
Queue memory	$O(R \times C)$ worst-case	$O(\min(R, C))$ typical

12. Common Patterns

Pattern	Identification	Approach	Example
Distance to nearest 0	"Distance to nearest 0"	Multi-source BFS from all 0s	LeetCode 542
Rotting oranges	"Minutes until all oranges rot"	Multi-source BFS with level tracking	LeetCode 994
As far from land as possible	"Max distance from any land"	Multi-source BFS from all land	LeetCode 1162
Map of highest peak	"Assign heights with distance from water"	Multi-source BFS from water	LeetCode 1765
Gate and walls	"Distance to nearest gate"	Multi-source BFS from all gates	LeetCode 286

13. Common Mistakes

- **Not initializing all sources:** Every source must be pushed to the queue with distance 0 before the BFS loop begins.
- **Double counting sources:** If a cell is both a source and visited later, its distance should remain 0.
- **Forgetting level tracking for time-based problems:** In Rotting Oranges, minutes correspond to BFS levels, not individual cells.
- **Not checking for unreachable cells:** Some cells may remain at INF distance if they cannot be reached from any source.
- **Modifying the grid unnecessarily:** Use a separate distance array instead of modifying the input.

14. Edge Cases

- No sources: All cells remain at INF distance
- All cells are sources: All distances are 0
- Single source: Degenerates to standard BFS
- Disconnected components: Cells in disconnected regions remain at INF
- Grid with only one cell: If source, distance 0; otherwise INF
- Large grid with many sources: BFS terminates quickly

15. Variations

Variation	Description	Importance
Multi-source with levels	Track time/steps as level count	High (LeetCode 994)
Multi-source with different source weights	Sources have different initial distances	Medium
Multi-source with obstacles	Some cells are blocked	Medium
Multi-source with 8-directional movement	Including diagonals	Medium

16. Related Algorithms/Data Structures

Related	Connection
Standard BFS	Multi-source is a generalization
Dijkstra	Multi-source Dijkstra for weighted graphs
Voronoi diagram	Multi-source BFS partitions grid by nearest source
Grassfire transform	Same as multi-source BFS in image processing

17. Practice Problems

Easy

- **Flood Fill** (LeetCode 733) — Single source, but related concept
- **Island Perimeter** (LeetCode 463) — Not BFS, but grid traversal

Medium

- **01 Matrix** (LeetCode 542) — Distance to nearest 0
- **Rotting Oranges** (LeetCode 994) — Multi-source BFS with levels
- **As Far from Land as Possible** (LeetCode 1162) — Max distance from land

Hard

- **Map of Highest Peak** (LeetCode 1765) — Multi-source BFS with height assignment
- **Shortest Path to Get All Keys** (LeetCode 864) — BFS with state (related)

18. Interview Explanation

"Multi-source BFS is used when we need the shortest distance from any cell to the nearest of multiple sources. Instead of running BFS from each source separately, we push all sources into the queue with distance 0 and run BFS once. This is equivalent to adding a virtual super-source connected to all actual sources. The algorithm is $O(R \times C)$ time and space. Common applications include computing distance to the nearest 0 in a binary matrix, simulating the spread of rotting oranges, and finding the maximum distance from any land cell."

19. Revision Notes

- Push all sources to queue with dist = 0
- Run standard BFS
- Each cell's distance = shortest distance to any source
- Level-by-level BFS for time-based problems
- $O(R \times C)$ time, $O(R \times C)$ space
- Equivalent to virtual super-source + BFS
- Common problems: 01 Matrix, Rotting Oranges

20. Final Cheat Sheet

Aspect	Details
When to use	"Distance to nearest X", simultaneous spread problems
Main operations	Push all sources to queue, BFS, distance update
Complexity	$O(R \times C)$ time, $O(R \times C)$ space
Key code	<code>for each source: dist[src]=0, q.push(src); while(!q.empty()){...}</code>
Edge cases	No sources (all INF), all sources (all 0), single source

11. DP ON GRID

1. Overview

Dynamic Programming (DP) on a grid involves solving optimization problems on a 2D grid by breaking them into overlapping subproblems. Each cell's value depends on previously computed values from neighboring cells. Common examples include minimum path sum, unique paths, and maximum square.

2. Intuition

Analogy: Imagine you're at the top-left of a grid and want to reach the bottom-right, moving only right or down. The number of ways to reach any cell is the sum of ways to reach the cell above it and the cell to its left. This is the classic DP on grid — the answer for a cell depends only on answers from previously computed cells.

Why it works: DP on grid works because the grid has an inherent topological order. For problems where movement is only right/down (or up/left), we can process cells in row-major order, and each cell's value depends only on cells that have already been computed.

Step-by-step reasoning:

1. Define `dp[i][j]` = answer for cell `(i, j)`.
2. Initialize base cases (first row, first column, or specific cells).
3. For each cell in a valid order:
 - Compute `dp[i][j]` using `dp[i-1][j]`, `dp[i][j-1]`, etc.
4. Return `dp[R-1][C-1]` or the maximum/minimum over all cells.

3. When to Use It

- When the problem asks for counting (number of ways to reach a cell)
- When the problem asks for minimum/maximum path sum
- When the problem asks for the largest square/rectangle of 1s
- When movement is constrained (right/down only)
- When optimal substructure exists (answer for cell depends on neighbors)

Trigger phrases: "minimum path sum", "number of ways", "unique paths", "largest square", "maximum sum submatrix", "DP on grid"

4. When Not to Use It

- When movement is unrestricted (up/down/left/right random) — use BFS/Dijkstra
- When the grid is very large and DP table doesn't fit in memory (use space optimization)
- When the problem is about connectivity (use DFS/BFS)
- When the graph has cycles and no topological order exists
- When the problem is NP-hard (e.g., longest path in general grid)

5. Core Concepts

Concept	Explanation	Why It Matters
State definition	<code>dp[i][j]</code> meaning depends on problem	Must be clearly defined first
Base case	Values for first row, first column, or specific cells	Foundation for all other computations
Transition	How <code>dp[i][j]</code> depends on previous cells	The core of the DP solution
Topological order	Order of processing cells (row-major)	Must ensure dependencies are computed first
Space optimization	Using 1D array instead of 2D	Reduces $O(R \times C)$ to $O(C)$ or $O(\min(R, C))$
Tabulation vs memoization	Bottom-up (table) vs top-down (recursion + cache)	Different implementation styles

6. Step-by-Step Algorithm

Minimum Path Sum (LeetCode 64):

1. Let `dp[i][j]` = minimum sum to reach (i, j) from $(0, 0)$.
2. Initialize: `dp[0][0] = grid[0][0]`.
3. Fill first row: `dp[0][j] = dp[0][j-1] + grid[0][j]`.
4. Fill first column: `dp[i][0] = dp[i-1][0] + grid[i][0]`.
5. For $i = 1$ to $R-1$, $j = 1$ to $C-1$:
 - `dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])`.

6. Return $dp[R-1][C-1]$.

Unique Paths (LeetCode 62):

1. $dp[i][j]$ = number of ways to reach (i, j) from $(0, 0)$.
2. First row: $dp[0][j] = 1$ (only one way: keep moving right).
3. First column: $dp[i][0] = 1$ (only one way: keep moving down).
4. For other cells: $dp[i][j] = dp[i-1][j] + dp[i][j-1]$.
5. Return $dp[R-1][C-1]$.

7. Dry Run

Minimum Path Sum: Grid:

```
1 3 1
1 5 1
4 2 1
```

Cell	$dp[i][j]$	Computation
(0,0)	1	Base
(0,1)	4	$1 + 3$
(0,2)	5	$4 + 1$
(1,0)	2	$1 + 1$
(1,1)	7	$1 + \min(2, 4) = 1 + 2 = 3$? Wait: $grid[1][1]=5$, $\min(dp[0][1]=4, dp[1][0]=2) = 2$, so $5+2=7$
(1,2)	8	$1 + \min(7, 5) = 1 + 5 = 6$? Wait: $grid[1][2]=1$, $\min(dp[0][2]=5, dp[1][1]=7) = 5$, so $1+5=6$
(2,0)	6	$4 + 2$
(2,1)	8	$2 + \min(7, 6) = 2+6=8$
(2,2)	9	$1 + \min(6, 8) = 1+6=7$

Wait, let me redo:

```
dp[0][0] = 1
dp[0][1] = 1 + 3 = 4
dp[0][2] = 4 + 1 = 5
dp[1][0] = 1 + 1 = 2
dp[1][1] = 5 + min(4, 2) = 5 + 2 = 7
dp[1][2] = 1 + min(5, 7) = 1 + 5 = 6
dp[2][0] = 4 + 2 = 6
dp[2][1] = 2 + min(7, 6) = 2 + 6 = 8
dp[2][2] = 1 + min(6, 8) = 1 + 6 = 7
```

Minimum path sum = 7

Path: (0,0) → (1,0) → (2,0) → (2,1) → (2,2) Values: 1 + 1 + 4 + 2 + 1 = 9? No, wait.

Let me recheck. The path (0,0)→(1,0)→(2,0)→(2,1)→(2,2) has sum: 1 + 1 + 4 + 2 + 1 = 9.

But $dp[2][2] = 7$. Let me find the path: (0,0)=1 → (0,1)=3 → (0,2)=1 → (1,2)=1 → (2,2)=1 Sum = 1 + 3 + 1 + 1 + 1 = 7 ✓

Let me trace dp for this path: $dp[0][0]=1$, $dp[0][1]=4$, $dp[0][2]=5$ $dp[1][2]=1+\min(dp[0][2]=5, dp[1][1]=7)=1+5=6$ $dp[2][2]=1+\min(dp[1][2]=6, dp[2][1]=8)=1+6=7$ ✓

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Minimum Path Sum (LeetCode 64)
// Time: O(RxC), Space: O(RxC) or O(C) optimized
int minPathSum(vector<vector<int>>& grid) {
    int R = grid.size(), C = grid[0].size();
    vector<vector<int>> dp(R, vector<int>(C));

    dp[0][0] = grid[0][0];

    // Fill first row
    for (int j = 1; j < C; j++)
        dp[0][j] = dp[0][j-1] + grid[0][j];

    // Fill first column
    for (int i = 1; i < R; i++)
        dp[i][0] = dp[i-1][0] + grid[i][0];

    // Fill rest
    for (int i = 1; i < R; i++)
        for (int j = 1; j < C; j++)
            dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1]);

    return dp[R-1][C-1];
}

// Space-optimized version (1D array)
int minPathSumOptimized(vector<vector<int>>& grid) {
    int R = grid.size(), C = grid[0].size();
    vector<int> dp(C);

    dp[0] = grid[0][0];
    for (int j = 1; j < C; j++)
        dp[j] = dp[j-1] + grid[0][j];

    for (int i = 1; i < R; i++) {
        dp[0] += grid[i][0]; // first column
        for (int j = 1; j < C; j++)
            dp[j] = grid[i][j] + min(dp[j], dp[j-1]);
    }
}
```

```

    }

    return dp[C-1];
}

// Unique Paths (LeetCode 62)
int uniquePaths(int R, int C) {
    vector<vector<int>> dp(R, vector<int>(C, 1));

    for (int i = 1; i < R; i++)
        for (int j = 1; j < C; j++)
            dp[i][j] = dp[i-1][j] + dp[i][j-1];

    return dp[R-1][C-1];
}

// Unique Paths with Obstacles (LeetCode 63)
int uniquePathsWithObstacles(vector<vector<int>>& grid) {
    int R = grid.size(), C = grid[0].size();
    if (grid[0][0] == 1) return 0;

    vector<vector<long long>> dp(R, vector<long long>(C, 0));
    dp[0][0] = 1;

    // Fill first row
    for (int j = 1; j < C; j++)
        dp[0][j] = (grid[0][j] == 0) ? dp[0][j-1] : 0;

    // Fill first column
    for (int i = 1; i < R; i++)
        dp[i][0] = (grid[i][0] == 0) ? dp[i-1][0] : 0;

    for (int i = 1; i < R; i++)
        for (int j = 1; j < C; j++)
            if (grid[i][j] == 0)
                dp[i][j] = dp[i-1][j] + dp[i][j-1];

    return dp[R-1][C-1];
}

// Maximal Square (LeetCode 221)
int maximalSquare(vector<vector<char>>& matrix) {
    int R = matrix.size(), C = matrix[0].size();
    vector<vector<int>> dp(R, vector<int>(C, 0));
    int maxSide = 0;

    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            if (matrix[i][j] == '1') {
                if (i == 0 || j == 0) {
                    dp[i][j] = 1;
                } else {
                    dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
                }
            }
        }
    }
}

```

```

        maxSide = max(maxSide, dp[i][j]);
    }
}
}
return maxSide * maxSide;
}

int main() {
    vector<vector<int>> grid = {{1,3,1},{1,5,1},{4,2,1}};
    cout << minPathSum(grid) << "\n"; // 7
    cout << minPathSumOptimized(grid) << "\n"; // 7
    cout << uniquePaths(3, 7) << "\n"; // 28
    return 0;
}

```

9. Python Implementation

```

from typing import List

def minPathSum(grid: List[List[int]]) -> int:
    R, C = len(grid), len(grid[0])
    dp = [[0] * C for _ in range(R)]

    dp[0][0] = grid[0][0]
    # First row
    for j in range(1, C):
        dp[0][j] = dp[0][j-1] + grid[0][j]
    # First column
    for i in range(1, R):
        dp[i][0] = dp[i-1][0] + grid[i][0]
    # Rest
    for i in range(1, R):
        for j in range(1, C):
            dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])

    return dp[R-1][C-1]

def minPathSumOptimized(grid: List[List[int]]) -> int:
    R, C = len(grid), len(grid[0])
    dp = [0] * C
    dp[0] = grid[0][0]
    for j in range(1, C):
        dp[j] = dp[j-1] + grid[0][j]
    for i in range(1, R):
        dp[0] += grid[i][0]
        for j in range(1, C):
            dp[j] = grid[i][j] + min(dp[j], dp[j-1])
    return dp[C-1]

def uniquePaths(R: int, C: int) -> int:
    dp = [[1] * C for _ in range(R)]

```

```

    for i in range(1, R):
        for j in range(1, C):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]
    return dp[R-1][C-1]

def maximalSquare(matrix: List[List[str]]) -> int:
    R, C = len(matrix), len(matrix[0])
    dp = [[0] * C for _ in range(R)]
    max_side = 0
    for i in range(R):
        for j in range(C):
            if matrix[i][j] == '1':
                if i == 0 or j == 0:
                    dp[i][j] = 1
                else:
                    dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
            max_side = max(max_side, dp[i][j])
    return max_side * max_side

grid = [[1,3,1],[1,5,1],[4,2,1]]
print(minPathSum(grid)) # 7
print(uniquePaths(3, 7)) # 28

```

10. Code Explanation

- **Minimum Path Sum:** $dp[i][j]$ represents the minimum sum to reach (i, j) from $(0, 0)$. The transition $dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$ accounts for coming from above or from the left.
- **Space optimization:** Instead of a 2D DP table, we use a 1D array of size C . When processing row i , $dp[j]$ stores $dp[i][j]$. Before updating, $dp[j]$ is $dp[i-1][j]$ (from previous row), and $dp[j-1]$ is $dp[i][j-1]$ (already updated in current row).
- **Unique Paths:** Base case: first row and first column have exactly 1 way to reach them. All other cells: ways = ways from above + ways from left.
- **Maximal Square:** $dp[i][j]$ = side length of the largest square ending at (i, j) . The transition $1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$ checks the three neighbors that would form a larger square.

11. Complexity Analysis

Problem	Time	Space (2D)	Space (optimized)
Minimum Path Sum	$O(R \times C)$	$O(R \times C)$	$O(C)$
Unique Paths	$O(R \times C)$	$O(R \times C)$	$O(C)$
Unique Paths w/ Obstacles	$O(R \times C)$	$O(R \times C)$	$O(C)$
Maximal Square	$O(R \times C)$	$O(R \times C)$	$O(C)$

12. Common Patterns

Pattern	Identification	Approach	Example
Minimum/Maximum path sum	"Min/max sum from top-left to bottom-right"	DP with min/max transition	LeetCode 64
Counting paths	"Number of ways to reach"	DP with sum transition	LeetCode 62
Largest square/rectangle	"Largest square of 1s"	DP with min of three neighbors	LeetCode 221
Dungeon game	"Minimum initial health"	DP from bottom-right to top-left	LeetCode 174
Triangle	"Minimum path sum in triangle"	DP from bottom to top	LeetCode 120

13. Common Mistakes

- **Wrong initialization:** First row and column must be initialized correctly. For minimum path sum, `dp[0][j] = dp[0][j-1] + grid[0][j]`, not `grid[0][j]`.
- **Integer overflow:** Use `long long` when counting paths (unique paths can be large).
- **Not handling obstacles:** For grid with obstacles, set `dp[i][j] = 0` for blocked cells.
- **Incorrect transition order:** Ensure dependencies are computed before the current cell.
- **Forgetting MOD:** In competitive programming, path counts may require modulo.
- **Space optimization bugs:** When using 1D DP, the order of updates matters.

14. Edge Cases

- 1×1 grid: Return the single cell value
- $1 \times R$ grid: Only one path, sum of all cells
- $R \times 1$ grid: Only one path, sum of all cells
- Grid with obstacles at start or end: Return 0
- Grid with all zeros or all ones
- Large values causing overflow

15. Variations

Variation	Description	Importance
Triangle DP	DP on triangle grid (adjacent rows have different widths)	High (LeetCode 120)
Dungeon Game	DP from bottom-right to top-left	High (LeetCode 174)
Maximum sum submatrix	Kadane's algorithm extended to 2D	High (LeetCode 363)
Minimum falling path sum	DP with downward movement only	Medium (LeetCode 931)

16. Related Algorithms/Data Structures

Related	Connection
Memoization (top-down DP)	Alternative to tabulation, recursion + cache
Kadane's algorithm	1D maximum subarray, extended to 2D for max submatrix
Prefix sum 2D	Used to compute submatrix sums efficiently
BFS/Dijkstra	For unrestricted movement (not just right/down)

17. Practice Problems

Easy

- **Minimum Path Sum** (LeetCode 64) — Classic DP on grid
- **Unique Paths** (LeetCode 62) — Count paths, combinatorics also possible

Medium

- **Unique Paths II** (LeetCode 63) — With obstacles
- **Maximal Square** (LeetCode 221) — Largest square of 1s
- **Minimum Falling Path Sum** (LeetCode 931) — Downward movement

Hard

- **Dungeon Game** (LeetCode 174) — Reverse DP for minimum health
- **Max Sum of Rectangle No Larger Than K** (LeetCode 363) — 2D Kadane

18. Interview Explanation

"DP on a grid is used when the problem has optimal substructure and the movement is constrained, typically right/down. I define $dp[i][j]$ as the answer for the subgrid ending at (i, j) , initialize the first row and column, then fill the rest using a transition that depends on $dp[i-1][j]$ and $dp[i][j-1]$. The time complexity is $O(R \times C)$ and space can be optimized to $O(C)$ using a 1D array. Common problems include minimum path sum, unique paths, and maximal square."

19. Revision Notes

- Define $dp[i][j]$ clearly first
- Initialize base cases (first row, first column)
- Transition: depends on $dp[i-1][j]$ and $dp[i][j-1]$
- Space optimization: 1D array of size C
- For obstacles: skip blocked cells ($dp = 0$)
- Use long long for counting problems
- $O(R \times C)$ time, $O(C)$ optimized space

20. Final Cheat Sheet

Aspect	Details
When to use	Right/down movement, min/max sum, counting paths, largest square
Main operations	DP table fill, min/max/sum transitions
Complexity	$O(R \times C)$ time, $O(C)$ optimized space
Key code	<code>dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])</code>
Edge cases	1×1 , $1 \times R$, $R \times 1$, obstacles at start/end, overflow

12. PREFIX SUM 2D

1. Overview

A 2D prefix sum (also called "integral image" or "summed-area table") is a preprocessing technique that allows $O(1)$ queries of the sum of any submatrix. It is a 2D generalization of the 1D prefix sum array.

2. Intuition

Analogy: Imagine you have a grid of numbers, and you want to quickly know the sum of any rectangular region. Instead of summing all cells in the rectangle each time ($O(R \times C)$ per query), you precompute a table where each cell (i, j) contains the sum of all cells from $(0, 0)$ to (i, j) .

Why it works: Using the inclusion-exclusion principle, the sum of any submatrix can be computed from 4 prefix sum values. This is similar to how you compute the area of a rectangle using coordinates of its corners.

Step-by-step reasoning:

1. Precompute `prefix[i+1][j+1]` = sum of all cells in `grid[0..i][0..j]`.
2. The sum of submatrix $(r1, c1)$ to $(r2, c2)$ is: `sum = prefix[r2+1][c2+1] - prefix[r1][c2+1] - prefix[r2+1][c1] + prefix[r1][c1]`

3. When to Use It

- When you need to query submatrix sums multiple times
- When computing the sum of every possible submatrix (e.g., in sliding window problems)
- When the problem asks for "sum of submatrix" or "range sum query"
- When computing the maximum sum submatrix (Kadane's 2D)
- When you need to compute statistics over rectangular regions (mean, variance)

Trigger phrases: "range sum query", "submatrix sum", "sum of rectangle", "prefix sum 2D", "integral image"

4. When Not to Use It

- When only a single query is needed (just sum the submatrix directly: $O(R \times C)$)
- When the grid is updated frequently (use Fenwick tree 2D or segment tree 2D)
- When the grid is extremely large and memory is limited ($O(R \times C)$ memory may be too much)
- When you need non-sum queries (product, XOR — use other prefix techniques)

5. Core Concepts

Concept	Explanation	Why It Matters
Prefix sum definition	$\text{pref}[i][j] = \text{sum of grid}[0..i-1][0..j-1]$	Uses 1-indexed for simpler formulas
Inclusion-exclusion	$\text{sum} = \text{pref}[r2+1][c2+1] - \text{pref}[r1][c2+1] - \text{pref}[r2+1][c1] + \text{pref}[r1][c1]$	Core formula for $O(1)$ queries
1-indexed padding	Adding an extra row and column of zeros at the top/left	Eliminates edge case checks
Preprocessing	$\text{pref}[i][j] = \text{grid}[i-1][j-1] + \text{pref}[i-1][j] + \text{pref}[i][j-1] - \text{pref}[i-1][j-1]$	Builds the prefix table in $O(R \times C)$

6. Step-by-Step Algorithm

Preprocessing:

1. Let $R = \text{rows}$, $C = \text{columns}$.
2. Create pref of size $(R+1) \times (C+1)$, initialized to 0.
3. For $i = 1$ to R , $j = 1$ to C :
 - $\text{pref}[i][j] = \text{grid}[i-1][j-1] + \text{pref}[i-1][j] + \text{pref}[i][j-1] - \text{pref}[i-1][j-1]$

Query submatrix sum ($r1, c1$) to ($r2, c2$):

1. $\text{sum} = \text{pref}[r2+1][c2+1] - \text{pref}[r1][c2+1] - \text{pref}[r2+1][c1] + \text{pref}[r1][c1]$

7. Dry Run

Grid:

```
1 2 3
4 5 6
7 8 9
```

Prefix table (1-indexed):

```
0 0 0 0
0 1 3 6
0 5 12 21
0 12 27 45
```

Query: Sum of submatrix (1,1) to (2,2) → cells: 5,6,8,9

```
sum = pref[3][3] - pref[1][3] - pref[3][1] + pref[1][1]
     = 45 - 6 - 12 + 1
```

= 28

Check: $5 + 6 + 8 + 9 = 28$ ✓

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class NumMatrix {
    vector<vector<int>> pref;
    int R, C;

public:
    NumMatrix(vector<vector<int>>& matrix) {
        if (matrix.empty() || matrix[0].empty()) {
            R = C = 0;
            return;
        }
        R = matrix.size();
        C = matrix[0].size();
        pref.assign(R + 1, vector<int>(C + 1, 0));

        // Build prefix sum
        for (int i = 1; i <= R; i++) {
            for (int j = 1; j <= C; j++) {
                pref[i][j] = matrix[i-1][j-1]
                    + pref[i-1][j]
                    + pref[i][j-1]
                    - pref[i-1][j-1];
            }
        }
    }

    // Query sum of submatrix (r1, c1) to (r2, c2) inclusive
    int sumRegion(int r1, int c1, int r2, int c2) {
        return pref[r2 + 1][c2 + 1]
            - pref[r1][c2 + 1]
            - pref[r2 + 1][c1]
            + pref[r1][c1];
    }
};

// Maximum sum submatrix (Kadane's 2D)
int maxSumSubmatrix(vector<vector<int>>& matrix, int k) {
    int R = matrix.size(), C = matrix[0].size();
    int maxSum = INT_MIN;

    // Fix left column
    for (int left = 0; left < C; left++) {
        vector<int> rowSum(R, 0);
```

```

        // Extend to right
        for (int right = left; right < C; right++) {
            // Add current column to row sums
            for (int i = 0; i < R; i++)
                rowSum[i] += matrix[i][right];

            // 1D Kadane on rowSum
            int cur = 0, best = INT_MIN;
            for (int x : rowSum) {
                cur = max(x, cur + x);
                best = max(best, cur);
            }
            maxSum = max(maxSum, best);
        }
    }
    return maxSum;
}

int main() {
    vector<vector<int>> matrix = {{1,2,3},{4,5,6},{7,8,9}};
    NumMatrix nm(matrix);
    cout << nm.sumRegion(1, 1, 2, 2) << "\n"; // 28

    // Also works for 0-indexed queries
    cout << nm.sumRegion(0, 0, 0, 0) << "\n"; // 1
    cout << nm.sumRegion(0, 0, 2, 2) << "\n"; // 45
    return 0;
}

```

9. Python Implementation

```

from typing import List

class NumMatrix:
    def __init__(self, matrix: List[List[int]]):
        if not matrix or not matrix[0]:
            self.pref = [[0]]
            return
        R, C = len(matrix), len(matrix[0])
        self.pref = [[0] * (C + 1) for _ in range(R + 1)]

        for i in range(1, R + 1):
            for j in range(1, C + 1):
                self.pref[i][j] = (matrix[i-1][j-1]
                                   + self.pref[i-1][j]
                                   + self.pref[i][j-1]
                                   - self.pref[i-1][j-1])

    def sumRegion(self, r1: int, c1: int, r2: int, c2: int) -> int:
        return (self.pref[r2 + 1][c2 + 1]
                - self.pref[r1][c2 + 1]

```

```

        - self.pref[r2 + 1][c1]
        + self.pref[r1][c1])

def maxSumSubmatrix(matrix: List[List[int]], k: int) -> int:
    R, C = len(matrix), len(matrix[0])
    max_sum = float('-inf')

    for left in range(C):
        row_sum = [0] * R
        for right in range(left, C):
            for i in range(R):
                row_sum[i] += matrix[i][right]

            # 1D Kadane
            cur = 0
            best = float('-inf')
            for x in row_sum:
                cur = max(x, cur + x)
                best = max(best, cur)
            max_sum = max(max_sum, best)

    return max_sum

matrix = [[1,2,3],[4,5,6],[7,8,9]]
nm = NumMatrix(matrix)
print(nm.sumRegion(1, 1, 2, 2)) # 28
print(nm.sumRegion(0, 0, 2, 2)) # 45

```

10. Code Explanation

- **Construction:** We build a $(R+1) \times (C+1)$ prefix table. The extra row and column of zeros avoid special-casing the boundaries. The formula `pref[i][j] = grid[i-1][j-1] + pref[i-1][j] + pref[i][j-1] - pref[i-1][j-1]` uses inclusion-exclusion to compute the sum of the rectangle from (0,0) to (i-1,j-1).
- **Query:** `sumRegion(r1, c1, r2, c2)` uses the inclusion-exclusion formula with the 1-indexed prefix table. The result is the sum of the submatrix bounded by the given coordinates.
- **Maximum sum submatrix:** Uses Kadane's algorithm extended to 2D. Fix left and right columns, accumulate row sums, and apply 1D Kadane on the row-sum array.

11. Complexity Analysis

Operation	Time	Space
Preprocessing	$O(R \times C)$	$O(R \times C)$
Submatrix sum query	$O(1)$	—
Max sum submatrix (Kadane 2D)	$O(R \times C^2)$ or $O(C \times R^2)$	$O(R)$ or $O(C)$

12. Common Patterns

Pattern	Identification	Approach	Example
Range sum query	"Sum of submatrix"	Prefix sum 2D	LeetCode 304
Max sum submatrix	"Maximum sum rectangle"	2D Kadane (prefix sum + 1D Kadane)	LeetCode 363
Count submatrices with sum	"Number of submatrices with sum = k"	Prefix sum + hashmap	LeetCode 1074
Matrix block sum	"Sum of all k×k blocks"	Prefix sum for each block	LeetCode 1314

13. Common Mistakes

- **Off-by-one in prefix indices:** The prefix table is 1-indexed, but queries use 0-indexed coordinates. Always add 1 when accessing the prefix table.
- **Wrong formula:** The inclusion-exclusion formula is specific. Missing a term gives wrong results.
- **Integer overflow:** Submatrix sums can exceed 32-bit integers. Use `long long` when needed.
- **Not handling empty matrix:** Check for empty input before building the prefix table.
- **Building the table incorrectly:** The formula `pref[i][j] = grid[i-1][j-1] + pref[i-1][j] + pref[i][j-1] - pref[i-1][j-1]` must be used correctly.

14. Edge Cases

- Empty matrix: Return 0 for all queries
- 1×1 matrix: Prefix table is 2×2
- Single row: Works fine
- Single column: Works fine
- Negative values: Formula still works
- Query covering entire matrix: Should return total sum
- Query with r1=r2, c1=c2 (single cell): Should return that cell's value

15. Variations

Variation	Description	Importance
Prefix product 2D	Product instead of sum	Low
Prefix XOR 2D	XOR instead of sum	Medium
Fenwick tree 2D	Supports point updates and range queries	High (for dynamic grids)
Difference array 2D	Supports range updates and point queries	Medium

16. Related Algorithms/Data Structures

Related	Connection
1D Prefix Sum	Foundation for 2D version

Related	Connection
Fenwick Tree (BIT) 2D	Dynamic version with point updates
Segment Tree 2D	More powerful, supports range updates and queries
Difference Array 2D	Inverse operation (range update, point query)

17. Practice Problems

Easy

- **Range Sum Query 2D - Immutable** (LeetCode 304) — Standard prefix sum 2D
- **Matrix Block Sum** (LeetCode 1314) — Sum of $k \times k$ blocks

Medium

- **Maximum Sum of 3 Non-Overlapping Subarrays** (LeetCode 689) — Uses prefix sums
- **Count Submatrices With All Ones** (LeetCode 1504) — Prefix sum + histogram

Hard

- **Max Sum of Rectangle No Larger Than K** (LeetCode 363) — 2D Kadane + prefix sum
- **Number of Submatrices That Sum to Target** (LeetCode 1074) — Prefix sum + hashmap

18. Interview Explanation

"A 2D prefix sum allows $O(1)$ submatrix sum queries after $O(R \times C)$ preprocessing. I build a $(R+1) \times (C+1)$ table where $\text{pref}[i][j]$ is the sum of all cells from $(0,0)$ to $(i-1,j-1)$. The formula uses inclusion-exclusion: $\text{pref}[i][j] = \text{grid}[i-1][j-1] + \text{pref}[i-1][j] + \text{pref}[i][j-1] - \text{pref}[i-1][j-1]$. To query a submatrix, I use $\text{pref}[r2+1][c2+1] - \text{pref}[r1][c2+1] - \text{pref}[r2+1][c1] + \text{pref}[r1][c1]$. The key insight is the 1-indexed padding simplifies boundary handling."

19. Revision Notes

- Precompute: $\text{pref}[i][j] = \text{grid}[i-1][j-1] + \text{pref}[i-1][j] + \text{pref}[i][j-1] - \text{pref}[i-1][j-1]$
- Query: $\text{sum} = \text{pref}[r2+1][c2+1] - \text{pref}[r1][c2+1] - \text{pref}[r2+1][c1] + \text{pref}[r1][c1]$
- Use $(R+1) \times (C+1)$ table with zero padding
- $O(R \times C)$ preprocessing, $O(1)$ query
- Extension: Kadane 2D for max sum submatrix
- Edge cases: empty matrix, single cell

20. Final Cheat Sheet

Aspect	Details
When to use	Multiple submatrix sum queries, max sum submatrix
Main operations	Preprocessing ($O(R \times C)$), query ($O(1)$)

Aspect	Details
Complexity	$O(R \times C)$ preprocess, $O(1)$ query, $O(R \times C)$ space
Key code	<pre>pref[i][j] = grid[i-1][j-1] + pref[i-1][j] + pref[i][j-1] - pref[i-1][j-1]</pre>
Edge cases	Empty, single cell, negative values, overflow

13. CONNECTED COMPONENTS IN GRID

1. Overview

Connected components in a grid (also called "blobs," "clusters," or "islands") are groups of cells with the same value that are connected via adjacency (4-directional or 8-directional). Finding and labeling connected components is a fundamental grid operation.

2. Intuition

Analogy: Imagine a map with different colored regions (countries). Each contiguous region of the same color is a connected component. The goal is to identify and label each distinct region.

Why it works: The grid is treated as a graph where each cell is a node and edges exist between adjacent cells of the same value. Finding connected components is equivalent to finding all vertices in each connected component of this graph. We can use DFS, BFS, or Union-Find (DSU) to label components.

Step-by-step reasoning:

1. Scan the grid cell by cell.
2. When an unvisited cell is found, assign it a new component ID.
3. Traverse all connected cells of the same value (using DFS/BFS) and assign them the same component ID.
4. Continue scanning.
5. After processing, each cell has a component ID, and we know the size of each component.

3. When to Use It

- When counting islands or distinct regions
- When finding the size of each connected region
- When labeling cells with their component ID
- When computing properties of each component (area, perimeter, bounding box)
- When merging or modifying components
- When preprocessing for other algorithms (e.g., DP on components)

Trigger phrases: "connected components", "label components", "find clusters", "region labeling", "connected cells", "same color region"

4. When Not to Use It

- When you only need to count components (DFS/BFS counting is simpler)
- When the grid is extremely large and DSU is needed for better performance
- When the grid is dynamic (cells change value over time) — use DSU with updates
- When you need to find components in a graph (not grid) — use standard graph algorithms

5. Core Concepts

Concept	Explanation	Why It Matters
Component ID	A unique integer assigned to each connected component	Used to label cells and refer to components
Component size	Number of cells in a component	Used for area calculations
Connectivity	4-directional (up/down/left/right) or 8-directional (includes diagonals)	Determines what "connected" means
DFS/BFS traversal	Used to explore all cells in a component	Standard approach for labeling
Union-Find (DSU)	Alternative approach using disjoint set union	Better for dynamic grids and merging components

6. Step-by-Step Algorithm

DFS-based component labeling:

1. Create `comp[R][C]` initialized to 0 (unvisited).
2. Initialize `compId = 0`.
3. For each cell `(r, c)`:
 - If `comp[r][c] == 0` and `grid[r][c]` is a valid cell:
 - Increment `compId`.
 - Call `dfs(r, c, compId)`.
4. Return `comp` array and `compId` (number of components).

`dfs(r, c, id)`:

1. If out of bounds or `comp[r][c] != 0` or `grid[r][c]` is not valid: return.
2. Set `comp[r][c] = id`.
3. Increment `size[id]`.
4. Recurse to all 4 (or 8) neighbors.

7. Dry Run

Grid:

```
1 1 0 0
1 0 0 1
```

```
0 0 1 1
0 1 1 0
```

Cell	Value	Action	compld	Component Array
(0,0)	1	Start DFS, label as 1	1	1 ??? / ??? / ??? / ???
(0,1)	1	DFS → label 1	1	1 1 ?? / ??? / ??? / ???
(1,0)	1	DFS → label 1	1	1 1 ?? / 1 ??? / ??? / ???
(0,2)	0	Skip	1	—
(0,3)	0	Skip	1	—
(1,3)	1	Start DFS, label as 2	2	1 1 0 0 / 1 0 0 2 / 0 0 ?? / 0 ???
(2,2)	1	DFS → label 2	2	(continued)
(2,3)	1	DFS → label 2	2	—
(3,1)	1	DFS → label 2	2	—
(3,2)	1	DFS → label 2	2	—

Final component array:

```
1 1 0 0
1 0 0 2
0 0 2 2
0 2 2 0
```

Components: 2 (component 1: size 3, component 2: size 5)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class ConnectedComponents {
    int R, C;
    vector<vector<int>> grid;
    vector<vector<int>> comp;
    vector<int> compSize;
    int compCount;

    int dr[4] = {-1, 1, 0, 0};
    int dc[4] = {0, 0, -1, 1};

    void dfs(int r, int c, int id) {
        if (r < 0 || r >= R || c < 0 || c >= C) return;

```

```

        if (comp[r][c] != 0 || grid[r][c] == 0) return;

        comp[r][c] = id;
        compSize[id]++;

        for (int k = 0; k < 4; k++)
            dfs(r + dr[k], c + dc[k], id);
    }

public:
    ConnectedComponents(vector<vector<int>>& g) : grid(g) {
        R = g.size();
        C = g[0].size();
        comp.assign(R, vector<int>(C, 0));
        compCount = 0;

        for (int i = 0; i < R; i++) {
            for (int j = 0; j < C; j++) {
                if (comp[i][j] == 0 && grid[i][j] == 1) {
                    compCount++;
                    compSize.push_back(0);
                    dfs(i, j, compCount);
                }
            }
        }
    }

    int getComponentCount() { return compCount; }
    int getComponentSize(int id) { return compSize[id]; }
    int getComponentId(int r, int c) { return comp[r][c]; }
    vector<int> getAllSizes() { return compSize; }
};

// BFS-based component labeling
vector<vector<int>> labelComponentsBFS(vector<vector<int>>& grid) {
    int R = grid.size(), C = grid[0].size();
    vector<vector<int>> comp(R, vector<int>(C, 0));
    vector<int> compSize;
    int compCount = 0;

    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};

    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            if (comp[i][j] == 0 && grid[i][j] == 1) {
                compCount++;
                int size = 0;
                queue<pair<int,int>> q;
                q.push({i, j});
                comp[i][j] = compCount;

                while (!q.empty()) {
                    auto [r, c] = q.front(); q.pop();

```

```

        size++;

        for (int k = 0; k < 4; k++) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr >= 0 && nr < R && nc >= 0 && nc < C &&
                comp[nr][nc] == 0 && grid[nr][nc] == 1) {
                comp[nr][nc] = compCount;
                q.push({nr, nc});
            }
        }
    }
    compSize.push_back(size);
}
}
return comp;
}

// DSU-based component labeling
class DSU {
    vector<int> parent, sz;
public:
    DSU(int n) {
        parent.resize(n);
        sz.resize(n, 1);
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        return parent[x] == x ? x : parent[x] = find(parent[x]);
    }
    void unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return;
        if (sz[a] < sz[b]) swap(a, b);
        parent[b] = a;
        sz[a] += sz[b];
    }
    int getSize(int x) { return sz[find(x)]; }
};

vector<vector<int>>> labelComponentsDSU(vector<vector<int>>& grid) {
    int R = grid.size(), C = grid[0].size();
    DSU dsu(R * C);

    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};

    auto id = [&](int r, int c) { return r * C + c; };

    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            if (grid[i][j] == 1) {
                for (int k = 0; k < 4; k++) {
                    int ni = i + dr[k], nj = j + dc[k];

```

```

        if (ni >= 0 && ni < R && nj >= 0 && nj < C && grid[ni][nj] ==
1) {
            dsu.unite(id(i, j), id(ni, nj));
        }
    }
}

// Assign component IDs
vector<vector<int>> comp(R, vector<int>(C, 0));
unordered_map<int, int> idToComp;
int compCount = 0;

for (int i = 0; i < R; i++) {
    for (int j = 0; j < C; j++) {
        if (grid[i][j] == 1) {
            int root = dsu.find(id(i, j));
            if (idToComp.find(root) == idToComp.end())
                idToComp[root] = ++compCount;
            comp[i][j] = idToComp[root];
        }
    }
}

return comp;
}

int main() {
    vector<vector<int>> grid = {
        {1,1,0,0},
        {1,0,0,1},
        {0,0,1,1},
        {0,1,1,0}
    };

    ConnectedComponents cc(grid);
    cout << "Number of components: " << cc.getComponentCount() << "\n"; // 2
    cout << "Component sizes: ";
    for (int s : cc.getAllSizes())
        cout << s << " "; // 3 5
    cout << "\n";
    return 0;
}

```

9. Python Implementation

```

from typing import List, Tuple
from collections import deque

class ConnectedComponents:
    def __init__(self, grid: List[List[int]]):

```

```

self.grid = grid
self.R = len(grid)
self.C = len(grid[0])
self.comp = [[0] * self.C for _ in range(self.R)]
self.comp_size = []
self.comp_count = 0
self.dirs = [(-1,0), (1,0), (0,-1), (0,1)]

for i in range(self.R):
    for j in range(self.C):
        if self.comp[i][j] == 0 and grid[i][j] == 1:
            self.comp_count += 1
            self.comp_size.append(0)
            self._dfs(i, j, self.comp_count)

def _dfs(self, r: int, c: int, id: int):
    if r < 0 or r >= self.R or c < 0 or c >= self.C:
        return
    if self.comp[r][c] != 0 or self.grid[r][c] == 0:
        return
    self.comp[r][c] = id
    self.comp_size[id] += 1
    for dr, dc in self.dirs:
        self._dfs(r + dr, c + dc, id)

def get_component_count(self) -> int:
    return self.comp_count

def get_component_size(self, id: int) -> int:
    return self.comp_size[id]

def get_component_id(self, r: int, c: int) -> int:
    return self.comp[r][c]

# BFS version
def label_components_bfs(grid: List[List[int]]) -> List[List[int]]:
    R, C = len(grid), len(grid[0])
    comp = [[0] * C for _ in range(R)]
    dirs = [(-1,0), (1,0), (0,-1), (0,1)]
    comp_count = 0

    for i in range(R):
        for j in range(C):
            if comp[i][j] == 0 and grid[i][j] == 1:
                comp_count += 1
                q = deque([(i, j)])
                comp[i][j] = comp_count

                while q:
                    r, c = q.popleft()
                    for dr, dc in dirs:
                        nr, nc = r + dr, c + dc
                        if 0 <= nr < R and 0 <= nc < C and comp[nr][nc] == 0 and
grid[nr][nc] == 1:

```

```

        comp[nr][nc] = comp_count
        q.append((nr, nc))

    return comp

grid = [[1,1,0,0],[1,0,0,1],[0,0,1,1],[0,1,1,0]]
cc = ConnectedComponents(grid)
print(f"Components: {cc.get_component_count()}") # 2
print(f"Sizes: {cc.comp_size[1:]}") # [3, 5]

```

10. Code Explanation

- **DFS-based labeling:** The `ConnectedComponents` class encapsulates the grid, component array, and component sizes. The constructor scans the grid and calls `dfs` for each unvisited valid cell.
- **DFS function:** Marks the current cell with the component ID, increments the size counter, and recurses to all 4 neighbors.
- **BFS-based labeling:** Uses a queue instead of recursion. Same logic but avoids stack overflow for large grids.
- **DSU-based labeling:** More complex but useful for dynamic scenarios. Each cell is a DSU element. Adjacent cells of the same value are united. After processing, all cells in the same component have the same root.

11. Complexity Analysis

Approach	Time	Space
DFS/BFS labeling	$O(R \times C)$	$O(R \times C)$ for comp array
DSU labeling	$O(R \times C \times \alpha(R \times C))$	$O(R \times C)$
Each component	$O(\text{size of component})$	$O(1)$ extra

12. Common Patterns

Pattern	Identification	Approach	Example
Component labeling	"Label each connected region"	DFS/BFS with component ID	LeetCode 200
Component size	"Find size of each region"	Track size during DFS/BFS	LeetCode 695
Component properties	"Perimeter, bounding box of each component"	Compute during traversal	LeetCode 463
Merge components	"Fill cell to connect components"	DSU or BFS from modified cell	LeetCode 827
Largest component	"Find largest connected region"	Track max size during traversal	LeetCode 695

13. Common Mistakes

- **Not marking visited before neighbors:** In DFS, mark the cell as visited before recursing to neighbors. Otherwise, neighbors may push the same cell multiple times.
- **Using wrong connectivity:** 4-directional vs 8-directional must match the problem specification.
- **Forgetting to reset visited array:** If running multiple queries, the visited array must be reset.
- **DSU index calculation:** $id = r * C + c$ must be correct. Forgetting $+$ c leads to collisions.
- **Stack overflow:** For large components, recursive DFS may overflow. Use BFS or iterative DFS.

14. Edge Cases

- Empty grid: 0 components
- Grid with no valid cells: 0 components
- Grid with all cells valid: 1 component
- Single cell: 1 component of size 1
- Single row or column: Works fine
- Checkerboard pattern (no adjacent same values): Each cell is its own component

15. Variations

Variation	Description	Importance
8-directional connectivity	Include diagonals as neighbors	Medium
Component with holes	Components can have holes (enclosed empty space)	Medium
Dynamic component tracking	Components change over time (add/remove cells)	Medium
Component statistics	Compute area, perimeter, centroid, bounding box	Medium

16. Related Algorithms/Data Structures

Related	Connection
DFS/BFS on grid	Core traversal for component labeling
Union-Find (DSU)	Alternative for static and dynamic components
Flood fill	Same as DFS/BFS for a single component
Number of Islands	Counting components (same problem)

17. Practice Problems

Easy

- **Number of Islands** (LeetCode 200) — Count connected components
- **Island Perimeter** (LeetCode 463) — Compute perimeter of component

Medium

- **Max Area of Island** (LeetCode 695) — Largest component size

- **Number of Distinct Islands** (LeetCode 694) — Unique shapes of components
- **Surrounded Regions** (LeetCode 130) — Boundary-connected components

Hard

- **Making A Large Island** (LeetCode 827) — Merge components by flipping one cell
- **Number of Islands II** (LeetCode 305) — Dynamic component addition

18. Interview Explanation

"Connected components on a grid are found by scanning every cell. When I find an unvisited cell of the target value, I increment the component counter and perform DFS or BFS to label all connected cells with the same component ID. I track the size of each component during traversal. This is essentially the same as finding connected components in a graph where each cell is a node and edges connect adjacent cells of the same value. The time complexity is $O(R \times C)$. For dynamic grids, I use Union-Find instead."

19. Revision Notes

- Scan grid, on unvisited valid cell → new component, DFS/BFS to label
- Component ID assigned to each cell in the same connected region
- Track size during traversal
- 4-directional (default) vs 8-directional
- DFS recursion may overflow → use BFS or iterative DFS
- DSU alternative for dynamic grids
- $O(R \times C)$ time, $O(R \times C)$ space

20. Final Cheat Sheet

Aspect	Details
When to use	Label regions, count clusters, find component sizes
Main operations	Scan, DFS/BFS from each unvisited cell, assign ID
Complexity	$O(R \times C)$ time, $O(R \times C)$ space
Key code	<code>if(!comp[r][c] && valid) { compId++; dfs(r,c,compId); }</code>
Edge cases	Empty, all valid, checkerboard, single cell

14. BITMASK GRID DP

1. Overview

Bitmask DP on a grid (also called "DP with state compression" or "DP with bitmask") is a technique where the state of a row or column is encoded as a bitmask. It is used when the number of columns is small (typically ≤ 20) and the state of each column can be represented by a single bit (0 or 1).

2. Intuition

Analogy: Imagine you're placing tiles on a chessboard. Each row has a fixed width (say 6 columns). The state of a row can be represented as a binary number where each bit indicates whether a cell is occupied or not. By iterating over all possible states ($2^6 = 64$ possibilities), we can find the optimal placement.

Why it works: When the grid has a small number of columns, we can represent the state of a row as a bitmask (an integer). The DP transition from one row to the next involves checking compatibility between the current row's mask and the previous row's mask. This allows us to solve problems that would otherwise be exponential in the grid size.

Step-by-step reasoning:

- 1. Enumerate all valid masks for a single row (e.g., no two adjacent bits set).
- 2. Define `dp[row][mask]` = optimal value for first `row` rows, with `mask` on the current row.
- 3. For each valid mask for the current row:
 - Check compatibility with the previous row's mask.
 - Update `dp[row][mask]` based on `dp[row-1][prevMask]` + value of current mask.

3. When to Use It

- When the number of columns (or rows) is small (≤ 20)
- When placing tiles, dominos, or objects on a grid with constraints
- When the problem involves "independent sets" or "no two adjacent" constraints
- When the state of a row can be encoded as a binary pattern
- When the problem is a variant of "maximum independent set" or "maximum sum" on a grid

Trigger phrases: "bitmask DP", "state compression", "dp with mask", "small columns", "tiling", "domino", "no two adjacent"

4. When Not to Use It

- When the grid dimensions are large (≥ 20 columns, 2^{20} is too large)
- When a simpler DP exists (e.g., standard DP on grid for right/down movement)
- When the constraints don't require bitmask optimization
- When the state cannot be encoded as a bitmask (e.g., multiple colors/values per cell)
- When the number of masks is too large (2^C is too big for $C > 20$)

5. Core Concepts

Concept	Explanation	Why It Matters
Bitmask	An integer where each bit represents a cell's state (0/1)	Encodes row state compactly
Valid masks	Masks that satisfy the problem's constraints (e.g., no two adjacent 1s)	Reduces the state space
Mask generation	Enumerating all valid masks via bit operations	Typically done once and reused

Concept	Explanation	Why It Matters
Compatibility check	Two masks (rows) don't conflict when placed adjacent	Ensures valid transitions
Transition	$dp[row][mask] = \max(dp[row - 1][prevMask] + value(row, mask))$	Core DP formula
Precomputation	Computing value/validity for each mask in each row	Speeds up DP transitions

6. Step-by-Step Algorithm

Maximum sum with no two adjacent cells (8-directional):

- Let `C = columns` (small, $\leq 10-15$).
- Generate all valid masks for a row: `mask & (mask << 1) == 0` (no horizontal adjacency).
- For each mask, compute `sum[mask] = sum of grid[row][col]` for bits set in mask.
- Initialize `dp[0][mask] = sum[mask]` for valid masks.
- For `row = 1` to `R-1`:
 - For each valid mask for current row:
 - For each valid mask for previous row:
 - If masks are compatible (no vertical/diagonal conflict):
 - $dp[row][mask] = \max(dp[row - 1][prevMask] + sum[mask])$
- Return `max(dp[R-1][mask])`.

7. Dry Run

Grid (2×3):

```
1 2 3
4 5 6
```

`C = 3`, so masks range from 0 to 7 (2^3).

Valid masks (no adjacent 1s): 0 (000), 1 (001), 2 (010), 4 (100), 5 (101)

Row 0 sums:

- mask 0: 0
- mask 1 (001): `grid[0][2] = 3`
- mask 2 (010): `grid[0][1] = 2`
- mask 4 (100): `grid[0][0] = 1`
- mask 5 (101): `grid[0][0] + grid[0][2] = 1 + 3 = 4`

dp[0]: `dp[0][0]=0, dp[0][1]=3, dp[0][2]=2, dp[0][4]=1, dp[0][5]=4`

Row 1 sums:

- mask 0: 0, mask 1: 6, mask 2: 5, mask 4: 4, mask 5: 10

Compatibility: Two masks conflict if they share a column (vertical) or adjacent columns (diagonal).

- mask 5 (101) conflicts with mask 1 (001) → shares column 2
- mask 5 (101) conflicts with mask 4 (100) → shares column 0
- mask 5 (101) compatible with mask 2 (010) → no conflict

$dp[1][5] = dp[0][2] + 10 = 2 + 10 = 12$

Checking all:

- $dp[1][0] = \max(dp[0][*]) = 4$
- $dp[1][1] = \max(dp[0][0], dp[0][2], dp[0][4]) + 6 = 4 + 6 = 10$
- $dp[1][2] = \max(dp[0][0], dp[0][1], dp[0][4]) + 5 = 4 + 5 = 9$
- $dp[1][4] = \max(dp[0][0], dp[0][1], dp[0][2]) + 4 = 3 + 4 = 7$
- $dp[1][5] = dp[0][2] + 10 = 2 + 10 = 12$

Maximum sum = 12 (row 0: mask 2 = cell (0,1)=2, row 1: mask 5 = cells (1,0)=4, (1,2)=6)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Maximum sum with no two adjacent cells (8-directional)
// Columns ≤ 15
int maxSumNoAdjacent(vector<vector<int>>& grid) {
    int R = grid.size(), C = grid[0].size();
    if (C > 15) return -1; // too large for bitmask

    // Generate all valid masks for a single row
    vector<int> validMasks;
    for (int mask = 0; mask < (1 << C); mask++) {
        if ((mask & (mask << 1)) == 0) // no adjacent 1s horizontally
            validMasks.push_back(mask);
    }
    int M = validMasks.size();

    // Precompute sum for each mask in each row
    vector<vector<int>> sum(R, vector<int>(M, 0));
    for (int i = 0; i < R; i++) {
        for (int j = 0; j < M; j++) {
            int mask = validMasks[j];
            for (int k = 0; k < C; k++) {
                if (mask & (1 << k))
                    sum[i][j] += grid[i][k];
            }
        }
    }

    // DP: dp[row][maskIndex]
    vector<vector<int>> dp(R, vector<int>(M, 0));
```

```

// Base case: first row
for (int j = 0; j < M; j++)
    dp[0][j] = sum[0][j];

// Fill remaining rows
for (int i = 1; i < R; i++) {
    for (int j = 0; j < M; j++) { // current row mask
        for (int k = 0; k < M; k++) { // previous row mask
            int curMask = validMasks[j];
            int prevMask = validMasks[k];

            // Check vertical conflict: same column
            if (curMask & prevMask) continue;
            // Check diagonal conflict: adjacent columns
            if ((curMask & (prevMask << 1)) || (curMask & (prevMask >> 1)))
                continue;

            dp[i][j] = max(dp[i][j], dp[i-1][k] + sum[i][j]);
        }
    }
}

// Answer is max over last row
return *max_element(dp[R-1].begin(), dp[R-1].end());
}

// Tiling problem: count ways to tile a 2xN board with 2x1 and 1x2 dominos
int countTilings(int N) {
    int C = 2; // 2 columns
    int states = 4; // 2^2 = 4 states

    // Precompute valid transitions
    // curr_mask -> next_mask is valid if the tiling is consistent
    vector<vector<int>> dp(N + 1, vector<int>(states, 0));
    dp[0][0] = 1; // base: no columns, no occupied cells

    for (int col = 0; col < N; col++) {
        for (int mask = 0; mask < states; mask++) {
            if (dp[col][mask] == 0) continue;

            // Try all possible placements for the current column
            if (mask == 0) {
                // Empty: place 2 vertical (1x2) dominos
                dp[col+1][0] += dp[col][0]; // place two 1x2 vertical
                // Place one 2x1 horizontal (top)
                dp[col+1][1] += dp[col][0]; // top cell occupied
                // Place one 2x1 horizontal (bottom)
                dp[col+1][2] += dp[col][0]; // bottom cell occupied
            } else if (mask == 1) {
                // Top cell occupied: place bottom horizontal 2x1
                dp[col+1][0] += dp[col][1]; // fills bottom and extends to next
            } else if (mask == 2) {
                // Bottom cell occupied: place top horizontal 2x1
                dp[col+1][0] += dp[col][2]; // fills top and extends to next
            }
        }
    }
}

```

```

    }
    // mask == 3: both occupied, skip (shouldn't happen)
  }
}

return dp[N][0]; // no pending cells after last column
}

int main() {
  vector<vector<int>> grid = {{1,2,3},{4,5,6}};
  cout << "Max sum no adjacent: " << maxSumNoAdjacent(grid) << "\n"; // 12

  cout << "Tilings of 2x12 board: " << countTilings(12) << "\n"; // 233
  return 0;
}

```

9. Python Implementation

```

from typing import List

def max_sum_no_adjacent(grid: List[List[int]]) -> int:
    R, C = len(grid), len(grid[0])
    if C > 15:
        return -1

    # Generate valid masks (no adjacent 1s horizontally)
    valid_masks = [m for m in range(1 << C) if (m & (m << 1)) == 0]
    M = len(valid_masks)

    # Precompute sum for each mask in each row
    row_sum = [[0] * M for _ in range(R)]
    for i in range(R):
        for j, mask in enumerate(valid_masks):
            s = 0
            for k in range(C):
                if mask & (1 << k):
                    s += grid[i][k]
            row_sum[i][j] = s

    # DP
    dp = [[0] * M for _ in range(R)]
    for j in range(M):
        dp[0][j] = row_sum[0][j]

    for i in range(1, R):
        for j, cur_mask in enumerate(valid_masks):
            for k, prev_mask in enumerate(valid_masks):
                # Vertical and diagonal conflicts
                if cur_mask & prev_mask:
                    continue
                if (cur_mask & (prev_mask << 1)) or (cur_mask & (prev_mask >> 1)):

```

```

        continue
        dp[i][j] = max(dp[i][j], dp[i-1][k] + row_sum[i][j])

    return max(dp[-1])

def count_tilings(N: int) -> int:
    # DP for 2xN board tiling with 2x1 and 1x2 dominos
    states = 4
    dp = [[0] * states for _ in range(N + 1)]
    dp[0][0] = 1

    for col in range(N):
        for mask in range(states):
            if dp[col][mask] == 0:
                continue
            if mask == 0:
                # Place two vertical 1x2, or one horizontal 2x1 (top or bottom)
                dp[col+1][0] += dp[col][0]
                dp[col+1][1] += dp[col][0]
                dp[col+1][2] += dp[col][0]
            elif mask == 1:
                dp[col+1][0] += dp[col][1]
            elif mask == 2:
                dp[col+1][0] += dp[col][2]

    return dp[N][0]

grid = [[1,2,3],[4,5,6]]
print(max_sum_no_adjacent(grid)) # 12
print(count_tilings(12)) # 233

```

10. Code Explanation

- **Valid mask generation:** `mask & (mask << 1) == 0` checks that no two adjacent bits are set. This ensures no two cells in the same row are adjacent horizontally.
- **Sum precomputation:** For each row and each valid mask, we compute the sum of grid cells where the mask has a 1 bit. This avoids recomputing during DP.
- **DP transition:** For each pair of masks (current and previous row), we check compatibility:
 - Vertical conflict: `curMask & prevMask` checks if any column has 1s in both rows.
 - Diagonal conflict: `curMask & (prevMask << 1)` and `curMask & (prevMask >> 1)` check if the mask has a 1 diagonally adjacent to a 1 in the previous row.
- **Tiling DP:** The state represents which cells in the current column are already occupied by a horizontal domino from the previous column. The transitions represent placing dominos of different orientations.

11. Complexity Analysis

Operation	Time	Space
Valid mask generation	$O(2^C)$	$O(2^C)$
Sum precomputation	$O(R \times M \times C)$	$O(R \times M)$

Operation	Time	Space
DP transition	$O(R \times M^2)$	$O(R \times M)$
Overall	$O(R \times 2^{(2C)} + \dots)$	$O(R \times 2^C)$

Where M = number of valid masks $\leq 2^C$.

12. Common Patterns

Pattern	Identification	Approach	Example
No adjacent cells	"Cannot select adjacent cells"	Bitmask with row-wise compatibility	LeetCode 1655
Tiling with dominos	"Place tiles on board"	Bitmask for column state	LeetCode 1240
Maximum independent set	"Select non-adjacent nodes in grid"	Bitmask DP on bipartite grid	Codeforces
Art gallery / museum	"Place guards with constraints"	Bitmask DP with row state	Codeforces
Broken chessboard	"Place knights/rooks with constraints"	Bitmask with transition rules	CSES

13. Common Mistakes

- **Wrong mask generation:** The conflict check `mask & (mask << 1)` is for horizontal adjacency. For different constraints, different checks are needed.
- **Too many masks:** If $C > 15-20$, 2^C is too large. Use meet-in-the-middle or other techniques.
- **Missing diagonal conflicts:** Many problems require checking both diagonal directions.
- **Integer overflow:** Use `long long` for counting problems.
- **Incorrect base case:** The first row has no previous row, so all valid masks are allowed.
- **Not precomputing sums:** Computing sum inside the inner loop adds unnecessary $O(C)$ factor.

14. Edge Cases

- Single row: Only one row, no compatibility check needed
- Single column: $C = 1$, masks are 0 and 1
- $C = 0$: Empty grid
- All cells blocked: Only mask 0 is valid
- Grid with $C > 15$: Bitmask DP is infeasible

15. Variations

Variation	Description	Importance
Profile DP	DP with column-by-column state (similar to bitmask)	High (see next section)

Variation	Description	Importance
Broken profile DP	Handling obstacles in the grid	Medium
Tiling with L-shaped trominos	More complex tile shapes	Medium
Maximum sum with constraints	Different adjacency constraints	Medium

16. Related Algorithms/Data Structures

Related	Connection
Profile DP	Column-by-column variant of bitmask DP
DP with bitmask	General technique for combinatorial optimization
Maximum independent set	Graph problem, bitmask DP on grid is a special case
Tiling / Domino	Classic application of bitmask DP

17. Practice Problems

Easy

- **Maximum Sum of 3 Non-Overlapping Subarrays** (LeetCode 689) — Related concept
- **House Robber** (LeetCode 198) — 1D no-adjacent (foundation)

Medium

- **Maximum Sum of 3 Non-Overlapping Subarrays** (LeetCode 689) — Sliding window
- **Domino and Tromino Tiling** (LeetCode 790) — $2 \times N$ tiling with DP

Hard

- **Tiling a Rectangle with the Fewest Squares** (LeetCode 1240) — Bitmask DP
- **Maximum Number of Events That Can Be Attended** (LeetCode 1353) — Scheduling with bitmask

18. Interview Explanation

"Bitmask DP on a grid is used when the number of columns is small (typically ≤ 15). I represent the state of each row as a bitmask — an integer where each bit indicates whether a cell is selected. I generate all valid masks (e.g., no adjacent 1s), precompute the value of each mask for each row, and then use DP where `dp[row][mask]` depends on compatible masks from the previous row. The time complexity is $O(R \times M^2)$ where M is the number of valid masks ($\leq 2^C$). This technique is used for tiling problems, independent set on grid, and placement problems."

19. Revision Notes

- $C \leq 15$ for feasible bitmask DP (2^C manageable)
- Valid masks: generated based on problem constraints

- Compatibility: check vertical and diagonal conflicts
- `dp[row][mask]` = optimal value for first `row` rows
- Precompute sums for each mask per row
- $O(R \times M^2)$ time, where M = number of valid masks
- Common constraints: no adjacent, no diagonal, tiling

20. Final Cheat Sheet

Aspect	Details
When to use	Small columns (≤ 15), placement/selection with constraints
Main operations	Mask generation, sum precomputation, DP transition with compatibility
Complexity	$O(R \times 2^{(2C)})$ worst-case, $O(R \times M^2)$ with pruning
Key code	<code>validMasks = [m for m in range(1<<C) if (m & (m<<1)) == 0]</code>
Edge cases	Single row, single column, C too large, all blocked

15. PROFILE DP

1. Overview

Profile DP (also called "DP with broken profile" or "plug DP") is a dynamic programming technique on grids where the state is a "profile" — a bitmask representing the boundary between processed and unprocessed cells. It is used for tiling problems, counting Hamiltonian paths, and other grid problems where the state transitions are column-by-column rather than row-by-row.

2. Intuition

Analogy: Imagine you're tiling a floor with dominos, working from left to right. The "profile" is the jagged edge between the tiles you've already placed and the empty space. As you place each tile, the profile changes. The DP tracks all possible profiles and counts how many ways each profile can be achieved.

Why it works: Profile DP processes cells one by one (or column by column), maintaining the state of the "boundary" between processed and unprocessed cells. This is a more fine-grained approach than row-by-row bitmask DP, allowing for more complex constraints.

Step-by-step reasoning:

1. Process cells in a specific order (typically row-major).
2. Maintain a bitmask representing the state of the cells on the boundary.
3. For each cell, try all possible placements/choices.
4. Update the profile based on the choice.
5. Accumulate DP values.

3. When to Use It

- When tiling a grid with dominoes, trominoes, or other shapes

- When counting the number of ways to fill a grid with tiles
- When the grid has obstacles (broken profile)
- When the problem requires more complex state transitions than row-by-row DP
- When the number of rows is small (typically $\leq 10-15$)

Trigger phrases: "tiling", "domino tiling", "profile DP", "broken profile", "plug DP", "paving", "count ways to tile"

4. When Not to Use It

- When a simple DP or formula exists (e.g., $2 \times N$ tiling has a closed form)
- When the grid is large (profile DP is exponential in the smaller dimension)
- When the problem is about optimization (sum, max) rather than counting
- When the shape is not a simple tiling (use general bitmask DP)

5. Core Concepts

Concept	Explanation	Why It Matters
Profile	A bitmask representing the state of the boundary cells	The key state in profile DP
Broken profile	Profile DP where the boundary is a zigzag line (column-by-column)	More general than row-by-row
Plug DP	An extension of profile DP for connectivity problems (Hamiltonian paths)	Advanced technique
Transition	How placing a tile changes the profile	The core of the DP recurrence
Cell order	Typically row-major, processing left-to-right, top-to-bottom	Defines the profile boundary

6. Step-by-Step Algorithm

Domino tiling (count ways to tile an $R \times C$ grid with 2×1 dominos):

1. Initialize $dp[0] = 1$ (empty profile before any cell).
2. For each cell (r, c) in row-major order:
 - For each profile $mask$:
 - If $dp[mask] == 0$, skip.
 - If the current cell is already occupied in the profile: transition to the next cell without placing a domino.
 - Else:
 - Place a horizontal domino (covers current and right cell): update profile.
 - Place a vertical domino (covers current and bottom cell): update profile.
3. After processing all cells, $dp[0]$ is the answer (no pending cells).

7. Dry Run

Tiling a 2×2 grid with 2×1 dominos: Let's use column-by-column profile DP.

States: $2^2 = 4$ (each bit represents whether a cell in the current column is already occupied by a horizontal domino from the previous column).

Column	Mask	Previous	Action	New Mask
0	0	dp[0]=1	Place vertical in col 0	0
0	0	dp[0]=1	Place horizontal (top and bottom)	0
0	0	dp[0]=1	Place horizontal (top only)	1 (top occupied in next col)
0	0	dp[0]=1	Place horizontal (bottom only)	2 (bottom occupied in next col)
1	0	dp[0][0]=1 (from vertical)	Complete	0
1	1	dp[0][1]=1 (from top only)	Complete bottom cell	0
1	2	dp[0][2]=1 (from bottom only)	Complete top cell	0

Total ways: dp[0] = 2 (two vertical dominos, or two horizontal dominos)

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Count ways to tile an RxC grid with 2x1 and 1x2 dominos
// Profile DP, cell by cell
long long countDominoTilings(int R, int C) {
    if (R * C % 2 != 0) return 0; // odd area, impossible
    if (R > C) swap(R, C); // ensure R is the smaller dimension

    int states = 1 << R;
    vector<long long> dp(states, 0);
    dp[0] = 1; // empty profile before starting

    // Process each cell in row-major order
    for (int c = 0; c < C; c++) {
        for (int r = 0; r < R; r++) {
            vector<long long> ndp(states, 0);
            int cell = 1 << r; // bit for current cell

            for (int mask = 0; mask < states; mask++) {
                if (dp[mask] == 0) continue;

                if (mask & cell) {
                    // Current cell already occupied (by a horizontal domino from
                    previous column)
                }
            }
        }
    }
}
```

```

        ndp[mask ^ cell] += dp[mask];
    } else {
        // Try placing a vertical domino (covers current and bottom
cell)
        if (r + 1 < R && !(mask & (cell << 1))) {
            ndp[mask] += dp[mask]; // vertical: no change to profile
        }
        // Try placing a horizontal domino (covers current and right
cell)
        if (c + 1 < C) {
            ndp[mask | cell] += dp[mask]; // mark cell for next column
        }
    }
    dp = move(ndp);
}
}

return dp[0]; // no pending cells
}

// Count ways to tile with monominoes (1x1) and dominos (2x1, 1x2)
long long countTilingsMonominoDomino(int R, int C) {
    if (R > C) swap(R, C);
    int states = 1 << R;
    vector<long long> dp(states, 0);
    dp[0] = 1;

    const long long MOD = 1e9 + 7;

    for (int c = 0; c < C; c++) {
        for (int r = 0; r < R; r++) {
            vector<long long> ndp(states, 0);
            int cell = 1 << r;

            for (int mask = 0; mask < states; mask++) {
                if (dp[mask] == 0) continue;
                long long val = dp[mask];

                if (mask & cell) {
                    // Cell occupied: move to next
                    ndp[mask ^ cell] = (ndp[mask ^ cell] + val) % MOD;
                } else {
                    // Place a monomino (1x1)
                    ndp[mask] = (ndp[mask] + val) % MOD;

                    // Place a vertical domino (2x1)
                    if (r + 1 < R && !(mask & (cell << 1))) {
                        ndp[mask] = (ndp[mask] + val) % MOD;
                    }

                    // Place a horizontal domino (1x2)
                    if (c + 1 < C) {
                        ndp[mask | cell] = (ndp[mask | cell] + val) % MOD;
                    }
                }
            }
        }
    }
}

```

```

        }
    }
    dp = move(ndp);
}

return dp[0];
}

int main() {
    cout << "2x2 tiling: " << countDominoTilings(2, 2) << "\n"; // 2
    cout << "2x3 tiling: " << countDominoTilings(2, 3) << "\n"; // 3
    cout << "3x3 tiling: " << countDominoTilings(3, 3) << "\n"; // 0 (odd area)
    cout << "4x4 tiling: " << countDominoTilings(4, 4) << "\n"; // 36
    return 0;
}

```

9. Python Implementation

```

from typing import List

def count_domino_tilings(R: int, C: int) -> int:
    if R * C % 2 != 0:
        return 0
    if R > C:
        R, C = C, R # ensure R is smaller

    states = 1 << R
    dp = [0] * states
    dp[0] = 1

    for c in range(C):
        for r in range(R):
            ndp = [0] * states
            cell = 1 << r

            for mask in range(states):
                if dp[mask] == 0:
                    continue
                val = dp[mask]

                if mask & cell:
                    # Cell occupied: move to next
                    ndp[mask ^ cell] += val
                else:
                    # Place vertical domino (covers current and bottom)
                    if r + 1 < R and not (mask & (cell << 1)):
                        ndp[mask] += val

                    # Place horizontal domino (covers current and right)

```

```

        if c + 1 < C:
            ndp[mask | cell] += val
        dp = ndp

    return dp[0]

def count_tilings_monomino_domino(R: int, C: int) -> int:
    MOD = 10**9 + 7
    if R > C:
        R, C = C, R

    states = 1 << R
    dp = [0] * states
    dp[0] = 1

    for c in range(C):
        for r in range(R):
            ndp = [0] * states
            cell = 1 << r

            for mask in range(states):
                if dp[mask] == 0:
                    continue
                val = dp[mask]

                if mask & cell:
                    ndp[mask ^ cell] = (ndp[mask ^ cell] + val) % MOD
                else:
                    # Monomino
                    ndp[mask] = (ndp[mask] + val) % MOD

                    # Vertical domino
                    if r + 1 < R and not (mask & (cell << 1)):
                        ndp[mask] = (ndp[mask] + val) % MOD

                    # Horizontal domino
                    if c + 1 < C:
                        ndp[mask | cell] = (ndp[mask | cell] + val) % MOD
            dp = ndp

    return dp[0]

print(f"2x2: {count_domino_tilings(2, 2)}") # 2
print(f"2x3: {count_domino_tilings(2, 3)}") # 3
print(f"4x4: {count_domino_tilings(4, 4)}") # 36

```

10. Code Explanation

- **State:** The profile is a bitmask of length R (number of rows). Bit *r* is 1 if cell (*r*, *c*) is already occupied by a horizontal domino from the previous column.
- **Cell processing:** We iterate over each cell in row-major order. For each cell, we consider all possible placements.

- **If cell is occupied** (`mask & cell`): We simply move to the next cell, clearing the bit in the new profile.
- **If cell is free**: We can place:
 - A vertical domino (covers current and cell below): no change to profile.
 - A horizontal domino (covers current and cell to the right): sets the bit for the current cell in the profile (marks it for the next column).
- **Optimization**: `if (R > C) swap(R, C)` ensures the smaller dimension is used for the profile, reducing the number of states.

11. Complexity Analysis

Operation	Time	Space
Profile DP (domino tiling)	$O(R \times C \times 2^R)$	$O(2^R)$
With monominoes	$O(R \times C \times 2^R)$	$O(2^R)$

12. Common Patterns

Pattern	Identification	Approach	Example
Domino tiling	"Tile with 2×1 dominos"	Profile DP, cell by cell	CSES
Monomino + domino	"Tile with 1×1 and 2×1 "	Same DP with extra option	Codeforces
Broken profile	"Grid with obstacles"	Skip blocked cells in DP	Codeforces
Tromino tiling	"Tile with L-shaped trominos"	More complex transitions	LeetCode 790
Plug DP	"Hamiltonian paths in grid"	Maintain connectivity of paths	Advanced

13. Common Mistakes

- **Wrong profile size**: The profile should be the smaller dimension. If $R > C$, swap them.
- **Not handling odd area**: If $R \times C$ is odd, tiling with 2×1 dominos is impossible.
- **Incorrect transition when cell is occupied**: The bit must be cleared in the transition.
- **Missing the "no operation" case**: When the cell is occupied, we must still transition (just move to the next cell).
- **Integer overflow**: Use `long long` or modulo for large counts.
- **Not resetting dp for each cell**: The DP must be updated cell by cell, not column by column.

14. Edge Cases

- $R \times C$ is odd: Return 0 (domino tiling)
- $R = 1$: Only horizontal dominos, answer is 1 if C is even
- $R = 2$: Fibonacci-like sequence
- Grid with obstacles: Blocked cells cannot have tiles placed on them
- Very large grid: Use matrix exponentiation for regular patterns

15. Variations

Variation	Description	Importance
Broken profile DP	Handle obstacles in the grid	High
Plug DP	Hamiltonian paths and cycles	Medium
Tiling with trominoes	L-shaped tiles	Medium
Tiling with multiple colors	Tiles have colors, need to avoid color conflicts	Low

16. Related Algorithms/Data Structures

Related	Connection
Bitmask Grid DP	Row-by-row version (less fine-grained)
Matrix Exponentiation	For tiling problems with regular patterns
Linear Recurrences	Some tiling problems have closed-form recurrences
Generating Functions	Theoretical approach to tiling problems

17. Practice Problems

Easy

- **Domino and Tromino Tiling** (LeetCode 790) — $2 \times N$ tiling, DP
- **Tiling a $2 \times N$ Board** (GFG) — Simple Fibonacci

Medium

- **Tiling a Grid With Dominos** (Codeforces) — Profile DP for $R \times C$
- **Broken Profile** (CSES) — Tiling with obstacles

Hard

- **Tiling a Rectangle with the Fewest Squares** (LeetCode 1240) — Bitmask DP
- **Plug DP** (Advanced) — Hamiltonian paths on grid

18. Interview Explanation

"Profile DP is used for tiling problems on a grid. I process cells one by one in row-major order and maintain a bitmask profile of size R (the smaller dimension). Each bit represents whether a cell in the current column is already occupied by a horizontal domino from the previous column. For each cell, I try all valid placements — vertical domino, horizontal domino, or monomino — and update the profile accordingly. The base case is an empty profile before any cell, and the answer is the number of ways to reach an empty profile after all cells. The time complexity is $O(R \times C \times 2^R)$."

19. Revision Notes

- Profile = bitmask of size R (smaller dimension)
- Process cells row-major, cell by cell

- If cell occupied (bit = 1): clear bit, move on
- If cell free: place vertical domino, horizontal domino, or monomino
- Swap R and C so $R \leq C$ (profile uses smaller dimension)
- Odd area → impossible for domino tiling
- Use `long long` or modulo for large counts
- $O(R \times C \times 2^R)$ time, $O(2^R)$ space

20. Final Cheat Sheet

Aspect	Details
When to use	Tiling problems, grid with small dimension
Main operations	Cell-by-cell processing, profile (bitmask) update
Complexity	$O(R \times C \times 2^R)$ time, $O(2^R)$ space
Key code	<pre>if(mask & cell) ndp[mask^cell] += dp[mask]; else { try vertical/horizontal }</pre>
Edge cases	Odd area (0), $1 \times N$, obstacles, $R > C$ (swap)

16. FINAL COMPARISON TABLE

Algorithm	Best For	Time	Space	Key Idea
Matrix Traversal	Visiting all cells	$O(R \times C)$	$O(1)$	Nested loops
Spiral Traversal	Boundary-by-boundary	$O(R \times C)$	$O(1)$	4 boundaries shrinking
Rotate Matrix	90° in-place	$O(R^2)$	$O(1)$	Transpose + reverse
Search in Sorted Matrix	$O(R+C)$ search	$O(R+C)$	$O(1)$	Top-right elimination
Flood Fill	Fill connected region	$O(N)$	$O(N)$	DFS/BFS from source
Number of Islands	Count components	$O(R \times C)$	$O(1)$	Scan + sink
BFS on Grid	Shortest path (unweighted)	$O(R \times C)$	$O(R \times C)$	Queue + dist array
DFS on Grid	Connectivity, paths	$O(R \times C)$	$O(R \times C)$	Recursion/stack
Shortest Path	Min steps/cost	$O(R \times C)$ to $O(R \times C \log(R \times C))$	$O(R \times C)$	BFS/Dijkstra/0-1 BFS
Multi-Source BFS	Nearest source distance	$O(R \times C)$	$O(R \times C)$	Queue with all sources

Algorithm	Best For	Time	Space	Key Idea
DP on Grid	Right/down optimization	$O(R \times C)$	$O(C)$	Table with transitions
Prefix Sum 2D	Submatrix sum queries	$O(R \times C)$ pre, $O(1)$ query	$O(R \times C)$	Inclusion-exclusion
Connected Components	Label regions	$O(R \times C)$	$O(R \times C)$	DFS/BFS with IDs
Bitmask Grid DP	Small columns, constraints	$O(R \times 2^{(2C)})$	$O(R \times 2^C)$	Row masks + compatibility
Profile DP	Tiling (small R)	$O(R \times C \times 2^R)$	$O(2^R)$	Cell-by-cell profile

Pro Tip: For any grid problem, first identify:

- 1. Is the grid unweighted or weighted? → BFS vs Dijkstra
- 2. Is movement restricted? → DP (right/down) vs BFS/DFS (all directions)
- 3. Is the number of columns small? → Bitmask/Profile DP
- 4. Do I need multiple queries? → Prefix sums
- 5. Am I counting or finding paths? → DP vs Traversal